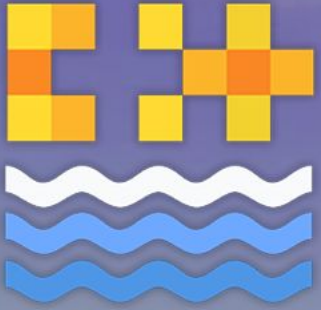




Fantastic Bugs and How to Test Them

Robert Leahy

2024

You should write tests

But why?



LSEG

Packet Capture

Packets arriving from the network have some length

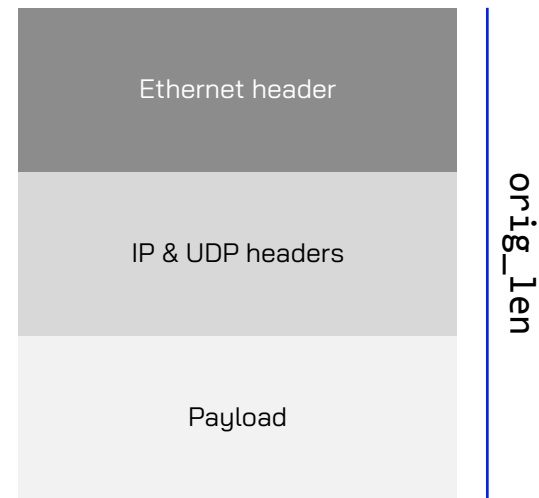
`orig_len`

Packets stored in the packet capture file have some length

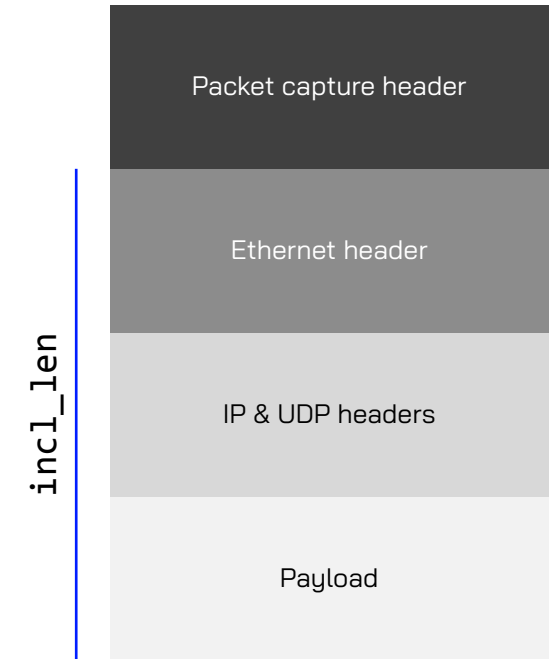
`incl_len`

These two lengths are sometimes the same, but sometimes different (i.e. the captured packet may be truncated or padded)

From network



In file



Size to Write vs. Size to Use

When manipulating a packet capture record in memory should only use up to the number of bytes actually captured

When writing the packet capture record back to a file should write exactly `incl_len` bytes

```
struct Entry {  
    // ...  
    std::size_t orig_len;  
    std::size_t incl_len;  
    constexpr std::size_t SizeToUse() const noexcept {  
        return std::min(orig_len, incl_len);  
    }  
    constexpr std::size_t SizeToWrite() const noexcept {  
        return std::min(orig_len, incl_len);  
    }  
    // ...  
};
```

Size to Write vs. Size to Use

When manipulating a packet capture record in memory should only use up to the number of bytes actually captured

When writing the packet capture record back to a file should write exactly `incl_len` bytes

Code has a bug

```
struct Entry {  
    // ...  
    std::size_t orig_len;  
    std::size_t incl_len;  
    constexpr std::size_t SizeToUse() const noexcept {  
        return std::min(orig_len, incl_len);  
    }  
    constexpr std::size_t SizeToWrite() const noexcept {  
        return std::min(orig_len, incl_len);  
    }  
    // ...  
};
```

Rewritten File is Corrupted

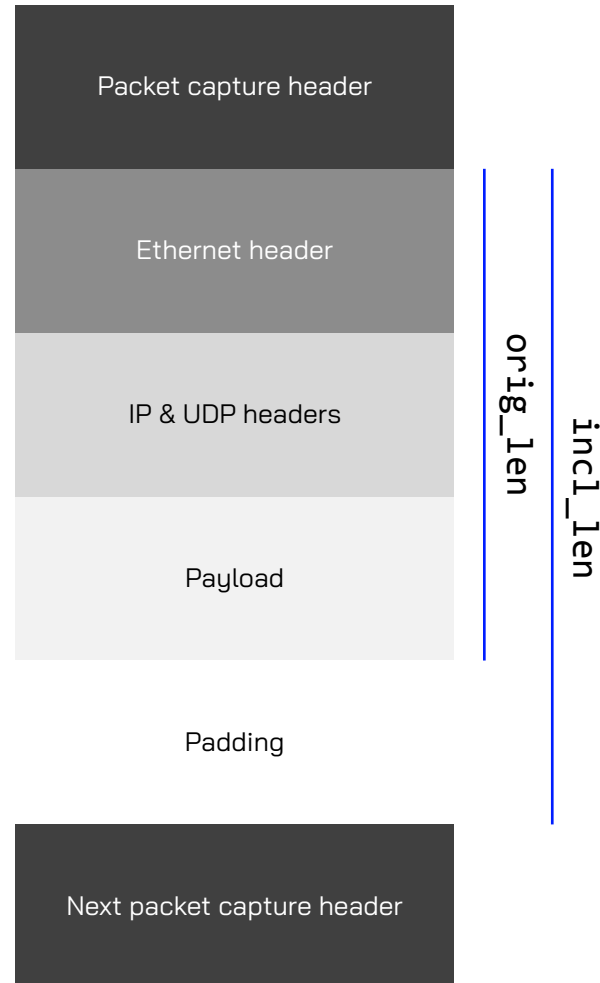
Rewritten file has `incl_len` which indicates padding is present, but padding isn't rewritten due to incorrect size to write

Reader regards part or all of subsequent packet capture header as part of padding

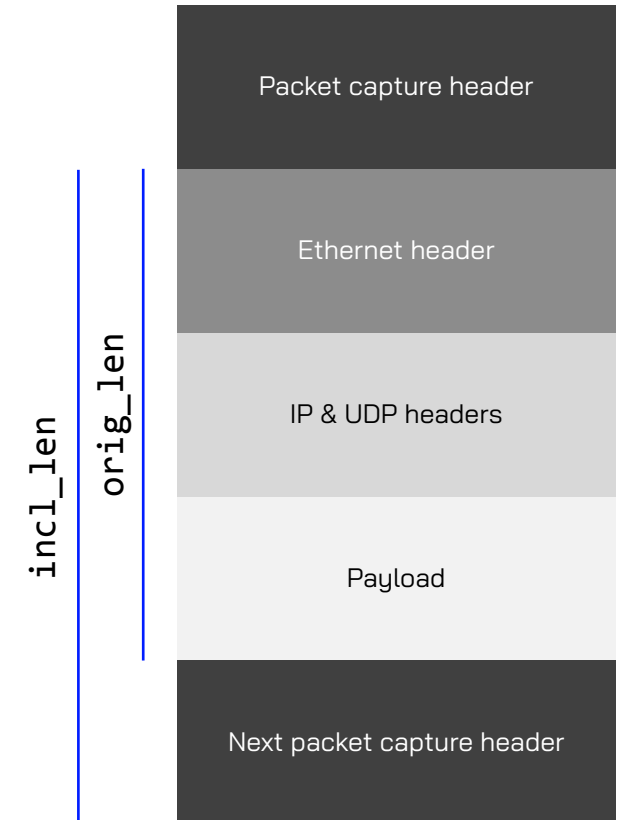
Reader regards bytes which aren't a packet capture header as being such a header

Data cannot be easily recovered

Original file



Rewritten file





Code is wrong by inspection



Just change it

No

commit ef03fc199d9ab93fa187929601bc968fff91a8ea

Author: Robert Leahy <robert@mayastreet.com>

Date: Fri May 31 12:35:48 2024 -0400

mstsplit: S3Sink & Padded Packets

bp::apps::mstsplit::S3Sink was added
(d216196f1d6a5a2f3ad5b90d14e087db4f3be499) after bp::apps::mstsplit::
Entry gained incl_len and orig_len members
(1d96802f1313a2f500d85c465b1121f28158cbc6, note also subsequent fix
aa734f791886f672cb12605e89175513550b36cc). Despite this when writing the
bodies of pcap entries bp::apps::mstsplit::S3Sink only wrote a number of
bytes equal to the value stored in bp::apps::mstsplit::Entry::size
(which is the minimum of incl_len and orig_len). This meant that when a
packet was padded (i.e. incl_len greater than orig_len) the number of
bytes in the body of the entry was not the number indicated in incl_len.
This caused consumers of the resulting pcap to be unable to properly
read it (as they would read past the end of the body).

Reproduced the above in a test and fixed.

Change-Id: Id201cc6708ad356d8705fc2e33071acb20e579e2

src/bp/apps/mstsplit/s3_sink.cpp | 2 +-
src/bp/apps/mstsplit/test/s3_sink_test.cpp | 150 ++++++

2 files changed, 151 insertions(+), 1 deletion(-)

commit ef03fc199d9ab93fa187929601bc968fff91a8ea

Author: Robert Leahy <robert@maystreet.com>

Date: Fri May 31 12:35:48 2024 -0400

mstsplit: S3Sink & Padded Packets

bp::apps::mstsplit::S3Sink was added
(d216196f1d6a5a2f3ad5b90d14e087db4f3be499) after bp::apps::mstsplit::
Entry gained incl_len and orig_len members
(1d96802f1313a2f500d85c465b1121f28158cbc6, note also subsequent fix
aa734f791886f672cb12605e89175513550b36cc). Despite this when writing the
bodies of pcap entries bp::apps::mstsplit::S3Sink only wrote a number of
bytes equal to the value stored in bp::apps::mstsplit::Entry::size
(which is the minimum of incl_len and orig_len). This meant that when a
packet was padded (i.e. incl_len greater than orig_len) the number of
bytes in the body of the entry was not the number indicated in incl_len.
This caused consumers of the resulting pcap to be unable to properly
read it (as they would read past the end of the body).

Reproduced the above in a test and fixed.

Change-Id: Id201cc6708ad356d8705fc2e33071acb20e579e2

src/bp/apps/mstsplit/s3_sink.cpp | 2 +-
src/bp/apps/mstsplit/test/s3_sink_test.cpp | 150 +++++
2 files changed, 151 insertions(+), 1 deletion(-)

Why write tests?

Generally we want to ensure code works, but precise reasons vary throughout the development cycle

Before we code

“Test-driven development”

Describe use cases and expectations thereof in code

Code starts out failing (obviously) and ends up passing

Often this step provides immediate feedback that design isn’t workable or convenient to use

As we code

Writing code facilitates consideration of edge cases and use cases not previously considered

These should be captured in tests to ensure they’re handled appropriately

After we code

Found or suspect a bug

When a test is written and identifies a bug it should be maintained since it has already been written

When a bug is found by inspection a test should still be written: Software systems are complex and looks can be deceiving, something that appears to be a bug may not be, or the “obvious” fix may have issues such that it doesn’t fully address the issue

Testing is About Systematizing Information

What does it mean for code to “work?”

Dividing code into “working” and “not working” requires some means of discriminating between the two

In a perfect universe every function would have a formal specification it implements, but our universe isn't perfect

Often what discriminates “working” and “not working” is discovered experientially
Experiences are ephemeral

Unit testing is a means of transcribing the lessons learned from this experiential journey and codifying what it means for something to “work”

Even if the universe was perfect and every function had a formal specification how would you determine whether or not the function actually implemented it?

You could prove it, or you could test it



Yeah it's some workaround for an edge case but there are no tests so I am trying to piece together the workaround

You Write Tests Anyway

You don't make changes to your code completely blind, do you?

Even if you don't write tests you:

- Write and run a throwaway application to see if your code is working
- Induce problematic behavior when you get a bug report
- Verify that fixes have changed code's behavior

How do you do all of the above?

- Write a throwaway executable, or
- Stand up an ad hoc environment (configuration, et cetera)

Those are tests

Instead:

- Write tests as you go
- Induce the problematic behavior in a test
- Verify that your test passes after your fix

Testing Captures the Fullness of Your Output

If you're going to do work make it count

If you go on an experiential journey but don't systemize the knowledge gained therefrom, you still spent time going on that journey

If you write a contrived configuration file and cobble together some corrupted files but delete it once the bug is fixed (without tests), you still spent time putting all that together

Why do work only to throw it away?

150 Lines?

Just to test 2?

Would be easy to write a test that checks the new value returned by `SizeToWrite`

But would that follow our three principles?

Systematize Information

Why should `SizeToUse` and `SizeToWrite` return different values under some circumstances?

Write the Tests You Were Going to Anyway

Despite only changing `SizeToWrite` I was surely going to test end-to-end, right?

Capture All Output

Author and commit a full end-to-end test: Read a padded pcap record, write it back, reread it, make sure that works

But how?

Rationalize the Filesystem

Morally the filesystem is a global, which makes it difficult from a testing point-of-view

With some care and attention it can be made sufficiently isolated that we can rigorously test against it

```
/**
 * \brief
 *   Securely create a uniquely named day exchange file
 *   database in somewhere suitable for temporary storage. It
 *   will be automatically destroyed upon destruction.
 */
result<day_exchange_file_database_ptr>
make_temporary_day_exchange_file_database(
    day_exchange_file_database_visitor* visitor = nullptr,
    llfio::handle::caching caching =
        llfio::handle::caching::temporary) noexcept;
```

commit 5895db45ab59591c9e6a0b6102a41e97764857c4

Author: Robert Leahy <robert@maystreet.com>

Date: Fri Apr 5 21:28:49 2024 -0400

bp::sys::PidFile: Use Temporary Directory in Unit Test

Since its addition in 8759f3b33fb0d649204d02f6583f5abc0d104e6b the sole unit test for bp::sys::PidFile has used a concrete path:

/var/tmp/pid_file_test.pid

Updated so it uses the temporary directory functionality which has been available in since f795adc0fcd8116e352fc627144f1e55973e2266.

Change-Id: Ie1cffeaaaf2047de489e3e62e13244e44f82d5bb7

commit a47e43fe7c80e68cf5a2140ae90549739e5cbe43

Author: Robert Leahy <robert@maystreet.com>

Date: Wed May 22 13:02:08 2024 -0400

bp::s3::MultipartUploadWithFallback: NFS Fallback

When writing the various files published to the fallback directory on failure bp::s3::MultipartUploadWithFallback previously did the following:

- If the file system supports O_TMPFILE:
 1. Create a temporary, unnamed file
 2. Write the desired contents
 3. Link the file into the file system at the destination
- Otherwise:
 1. Create a file with an extension indicating it is being written
 2. Write the desired contents
 3. Rename the file to the destination

By way of bp::sys::FileUtils::RenameFileNoReplace (ab681fd0fe9aa0279d8298091ca88105917babd5) step three of the latter protocol transitively used the renameat2 syscall with the RENAME_NOREPLACE flag if the system supported that syscall. Unfortunately whether or not the system supports renameat2 NFS does not support RENAME_NOREPLACE. This caused writing fallback files to fail with, for example:

Failed to rename .../3.part.inprogress to .../3.part: Invalid argument
Updated the second protocol described above replacing step three with:

3. Link the file at the destination
4. Unlink the original

Which was confirmed to work against NFS by running the newly-added unit test for this implementation detail against an NFS location.

commit a47e43fe7c80e68cf5a2140ae90549739e5cbe43

Author: Robert Leahy <robert@maystreet.com>

Date: Wed May 22 13:02:08 2024 -0400

bp::s3::MultipartUploadWithFallback: NFS Fallback

When writing the various files published to the fallback directory on failure bp::s3::MultipartUploadWithFallback previously did the following:

- If the file system supports O_TMPFILE:
 1. Create a temporary, unnamed file
 2. Write the desired contents
 3. Link the file into the file system at the destination
- Otherwise:
 1. Create a file with an extension indicating it is being written
 2. Write the desired contents
 3. Rename the file to the destination

By way of bp::sys::FileUtils::RenameFileNoReplace (ab681fd0fe9aa0279d8298091ca88105917babd5) step three of the latter protocol transitively used the renameat2 syscall with the RENAME_NOREPLACE flag if the system supported that syscall. Unfortunately whether or not the system supports renameat2 NFS does not support RENAME_NOREPLACE. This caused writing fallback files to fail with, for example:

Failed to rename .../3.part.inprogress to .../3.part: Invalid argument
Updated the second protocol described above replacing step three with:

3. Link the file at the destination
4. Unlink the original

Which was confirmed to work against NFS by running the newly-added unit test for this implementation detail against an NFS location.

Test

WriteFallback was previously a function with internal linkage in a source file

Put it in a header in a detail namespace for the sake of writing this test, but still in a detail namespace so it isn't part of the API provided to clients

This unit test passed when it was added, because tmpdir doesn't hand out directories on an NFS filesystem

```
TEST(MultipartUploadWithFallback, WriteFallback) {
    const std::string_view expected("foo");
    // ???
    const auto path = tmpdir / "foo.txt";
    EXPECT_FALSE(boost::filesystem::exists(path));
    detail::multipart_upload_with_fallback::WriteFallback(
        tmpdir,
        "foo.txt",
        expected.data(),
        expected.size());
    EXPECT_EQ(
        expected.size(),
        boost::filesystem::file_size(path));
    std::ifstream in(path.string());
    ASSERT_TRUE(in);
    std::string line;
    ASSERT_TRUE(std::getline(in, line));
    EXPECT_EQ(expected, line);
}
```

Mocking the Filesystem

tmpdir materializes an empty directory per test case

After the test case exits the directory is deleted

```
TEST(MultipartUploadWithFallback, WriteFallback) {
    const std::string_view expected("foo");
    // ???
    const auto path tmpdir "foo.txt";
    EXPECT_FALSE(boost::filesystem::exists(path));
    detail::multipart_upload_with_fallback::WriteFallback(
        tmpdir,
        "foo.txt",
        expected.data(),
        expected.size());
    EXPECT_EQ(
        expected.size(),
        boost::filesystem::file_size(path));
    std::ifstream in(path.string());
    ASSERT_TRUE(in);
    std::string line;
    ASSERT_TRUE(std::getline(in, line));
    EXPECT_EQ(expected, line);
}
```

Ad Hoc Change

Should likely build out some bona fide way to obtain a temporary test directory backed by a particular kind of filesystem

Fortunately (?) my home directory is backed by NFS

This ad hoc modification (not committed):

- Reproduced the issue
- Verified the fix
- Allowed all but this one line to be committed (and therefore systematized)

```
TEST(MultipartUploadWithFallback, WriteFallback) {  
    const std::string_view expected("foo");  
    const boost::filesystem::path tmpdir("~");  
    const auto path = tmpdir / "foo.txt";  
    EXPECT_FALSE(boost::filesystem::exists(path));  
    detail::multipart_upload_with_fallback::WriteFallback(  
        tmpdir,  
        "foo.txt",  
        expected.data(),  
        expected.size());  
    EXPECT_EQ(  
        expected.size(),  
        boost::filesystem::file_size(path));  
    std::ifstream in(path.string());  
    ASSERT_TRUE(in);  
    std::string line;  
    ASSERT_TRUE(std::getline(in, line));  
    EXPECT_EQ(expected, line);  
}
```

commit d11637dbcedeebba1cfc8a42b511d4586572e3c6

Author: Robert Leahy <robert@maystreet.com>

Date: Wed Mar 15 19:56:05 2023 +0000

SolarflareRecvGroup::ZeroCopyNext: Testing & Fixes

There has recently been quite a bit of churn in the implementation of bp::core::SolarflareRecvGroup::ZeroCopyNext:

- 9a2667fa9204c259c57d4906c740a480ec73fcf1
- 6fefab49442265ab016f666210309811e09bd216

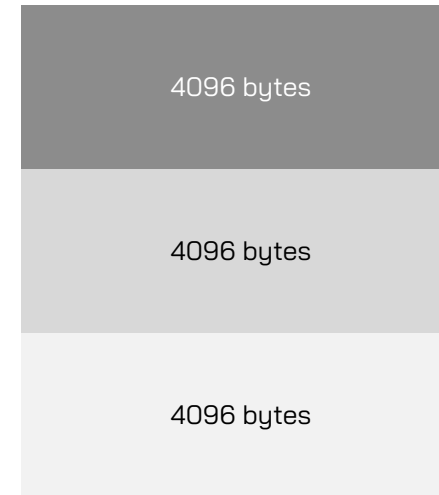
The fact that issues are being discovered and ostensibly corrected therein raises the concern that perhaps this member function requires more test coverage. Unfortunately being that it interacts directly with hardware in a performance-sensitive manner this is a delicate balancing act.

The core component which the aforementioned commits sought to modify is the control of the main loop of the aforementioned member function. As such extracted loop control from bp::core::SolarflareRecvGroup::ZeroCopyNext into a global function bp::core::detail::solarflare_recv_group::ZeroCopyNextImpl and added tests therefor. In the process thereof identified a bug which, for certain small timeouts (seemingly between 100 and 131 nanoseconds, inclusive) caused a variation of the bug purportedly addressed by 6fefab49442265ab016f666210309811e09bd216 to manifest.

Not only does this commit bring a critical but previously-untested component under test but it also lays the groundwork for iterating on the performance-sensitive algorithm which lies at the heart of bp::core::SolarflareRecvGroup::ZeroCopyNext: Now that the loop control algorithm is under test it can be iterated on, tweaked, et cetera with automated testing to ensure its ongoing correctness.

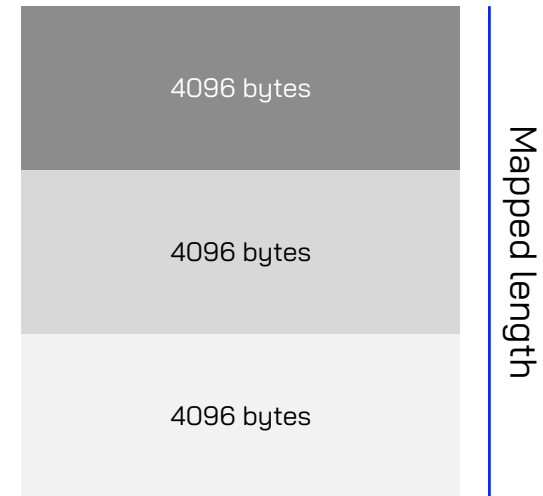
File

Total length is 12 288 bytes



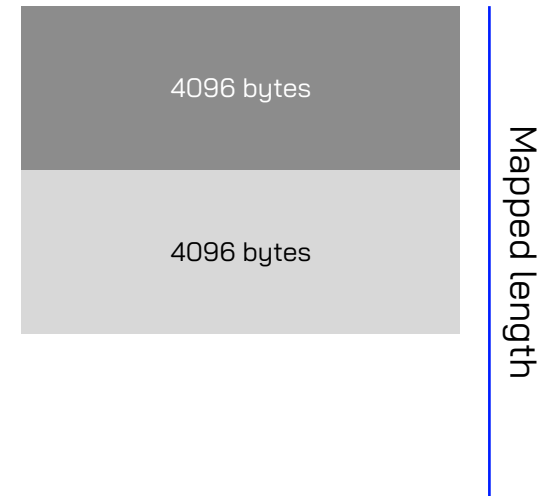
Memory Mapped File

The entire length of the file is mapped into memory



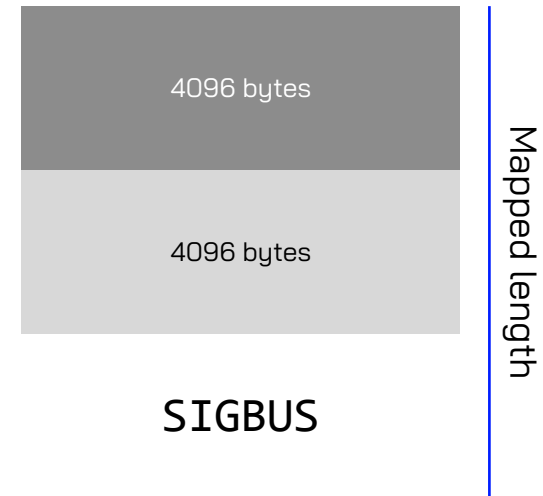
Truncate

Another user truncates the file to 8 192 bytes while it's mapped



SIGBUS

Any attempt to access the final page (which is no longer backed by the file) results in SIGBUS (whose default action is to terminate the program)



Memory Mapping a New File

Rather than mapping an
existing file we want to map a
new file every time

If the file already exists we
want to just overwrite it

```
struct MemoryMappedFile {  
    explicit MemoryMappedFile(const std::string& filename) : file_size_(0) {  
        // ...  
        fd_ = ::open(filename.c_str(), O_CREAT | O_TRUNC | O_RDWR, 0644);  
        // ...  
    }  
    void* Map(const MapStruct& map_struct) {  
        const auto required = map_struct->offset + map_struct->length;  
        if (  
            (required > file_size_) &&  
            (::ftruncate(fd_, map_struct->offset + map_struct->length) == -1))  
        {  
            file_size_ = required;  
            // ...  
        }  
        const auto retr = ::mmap(  
            nullptr,  
            map_struct->length,  
            PROT_READ | PROT_WRITE,  
            MAP_SHARED,  
            fd_,  
            map_struct->offset);  
        // ...  
        return retr;  
    }  
    // ...  
};
```

The Problem

open as we're calling it doesn't
always create a new file

If the file doesn't exist it
creates it

Otherwise it opens the existing
file and truncates it

If another process has the file
mapped when it tries to access
any of the bytes thereof it'll
receive SIGBUS

```
struct MemoryMappedFile {  
    explicit MemoryMappedFile(const std::string& filename) : file_size_(0) {  
        // ...  
        fd_ = ::open(filename.c_str(), O_CREAT | O_TRUNC O_RDWR, 0644);  
        // ...  
    }  
    void* Map(const MapStruct& map_struct) {  
        const auto required = map_struct->offset + map_struct->length;  
        if (  
            (required > file_size_) &&  
            (::ftruncate(fd_, map_struct->offset + map_struct->length) == -1))  
        {  
            file_size_ = required;  
            // ...  
        }  
        const auto retr = ::mmap(  
            nullptr,  
            map_struct->length,  
            PROT_READ | PROT_WRITE,  
            MAP_SHARED,  
            fd_,  
            map_struct->offset);  
        // ...  
        return retr;  
    }  
    // ...  
};
```

An Attempt Was Made

This addition of a blind unlink came without any tests

It addressed the problem in most instances, but not in general since it introduced a race condition

```
struct MemoryMappedFile {
    explicit MemoryMappedFile(const std::string& filename) : file_size_(0) {
        ::unlink(filename.c_str());
        fd_ = open(filename.c_str(), O_CREAT | O_TRUNC | O_RDWR, 0644);
        // ...
    }
    void* Map(const MapStruct& map_struct) {
        const auto required = map_struct->offset + map_struct->length;
        if (
            (required > file_size_) &&
            (::ftruncate(fd_, map_struct->offset + map_struct->length) == -1))
        {
            file_size_ = required;
            // ...
        }
        const auto retr = ::mmap(
            nullptr,
            map_struct->length,
            PROT_READ | PROT_WRITE,
            MAP_SHARED,
            fd_,
            map_struct->offset);
        // ...
        return retr;
    }
    // ...
};
```

Race Condition

Two processes may attempt to unlink, both find nothing to unlink, and then both call open, thereafter they will have the same file open (and one will have truncated the other)

```
struct MemoryMappedFile {
    explicit MemoryMappedFile(const std::string& filename) : file_size_(0) {
        ::unlink(filename.c_str());
        fd_ = ::open(filename.c_str(), O_CREAT | O_TRUNC | O_RDWR, 0644);
        // ...
    }
    void* Map(const MapStruct& map_struct) {
        const auto required = map_struct->offset + map_struct->length;
        if (
            (required > file_size_) &&
            (::ftruncate(fd_, map_struct->offset + map_struct->length) == -1))
        {
            file_size_ = required;
            // ...
        }
        const auto retr = ::mmap(
            nullptr,
            map_struct->length,
            PROT_READ | PROT_WRITE,
            MAP_SHARED,
            fd_,
            map_struct->offset);
        // ...
        return retr;
    }
    // ...
};
```

Process A

Race Condition

Two processes may attempt to unlink, both find nothing to unlink, and then both call open, thereafter they will have the same file open (and one will have truncated the other)

```
struct MemoryMappedFile {
    explicit MemoryMappedFile(const std::string& filename) : file_size_(0) {
        ::unlink(filename.c_str());
        fd_ = ::open(filename.c_str(), O_CREAT | O_TRUNC | O_RDWR, 0644)
        // ...
    }
    void* Map(const MapStruct& map_struct) {
        const auto required = map_struct->offset + map_struct->length;
        if (
            (required > file_size_) &&
            (::ftruncate(fd_, map_struct->offset + map_struct->length) == -1))
        {
            file_size_ = required;
            // ...
        }
        const auto retr = ::mmap(
            nullptr,
            map_struct->length,
            PROT_READ | PROT_WRITE,
            MAP_SHARED,
            fd_,
            map_struct->offset);
        // ...
        return retr;
    }
    // ...
};
```

Process A

Race Condition

Two processes may attempt to unlink, both find nothing to unlink, and then both call open, thereafter they will have the same file open (and one will have truncated the other)

```
struct MemoryMappedFile {
    explicit MemoryMappedFile(const std::string& filename) : file_size_(0) {
        ::unlink(filename.c_str());
        fd_ = ::open(filename.c_str(), O_CREAT | O_TRUNC | O_RDWR, 0644);
        // ...
    }
    void* Map(const MapStruct& map_struct) {
        const auto required = map_struct->offset + map_struct->length;
        if (
            (required > file_size_) &&
            (::ftruncate(fd_, map_struct->offset + map_struct->length) == -1))
        {
            file_size_ = required;
            // ...
        }
        const auto retr = ::mmap(
            nullptr,
            map_struct->length,
            PROT_READ | PROT_WRITE,
            MAP_SHARED,
            fd_,
            map_struct->offset);
        // ...
        return retr;
    }
    // ...
};
```

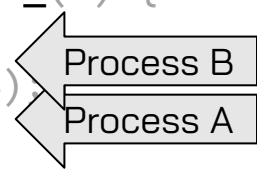
Diagram illustrating the race condition:

- Process B is shown with an arrow pointing to the `unlink` call.
- Process A is shown with an arrow pointing to the `open` call.

Race Condition

Two processes may attempt to unlink, both find nothing to unlink, and then both call open, thereafter they will have the same file open (and one will have truncated the other)

```
struct MemoryMappedFile {
    explicit MemoryMappedFile(const std::string& filename) : file_size_(0) {
        ::unlink(filename.c_str());
        fd_ = ::open(filename.c_str(), O_CREAT | O_TRUNC | O_RDWR, 0644);
        // ...
    }
    void* Map(const MapStruct& map_struct) {
        const auto required = map_struct->offset + map_struct->length;
        if (
            (required > file_size_) &&
            (::ftruncate(fd_, map_struct->offset + map_struct->length) == -1))
        {
            file_size_ = required;
            // ...
        }
        const auto retr = ::mmap(
            nullptr,
            map_struct->length,
            PROT_READ | PROT_WRITE,
            MAP_SHARED,
            fd_,
            map_struct->offset);
        // ...
        return retr;
    }
    // ...
};
```



The diagram shows two grey boxes labeled 'Process B' and 'Process A' with arrows pointing to the '0644' value in the code line 'fd_ = ::open(filename.c_str(), O_CREAT | O_TRUNC | O_RDWR, 0644);'. This illustrates that both processes are attempting to open the file with the same permissions, which is a key part of the race condition scenario.

Race Condition

Two processes may attempt to unlink, both find nothing to unlink, and then both call open, thereafter they will have the same file open (and one will have truncated the other)

```
struct MemoryMappedFile {
    explicit MemoryMappedFile(const std::string& filename) : file_size_(0) {
        ::unlink(filename.c_str());
        fd_ = ::open(filename.c_str(), O_CREAT | O_TRUNC | O_RDWR, 0644)
        // ...
    }
    void* Map(const MapStruct& map_struct) {
        const auto required = map_struct->offset + map_struct->length;
        if (
            (required > file_size_) &&
            (::ftruncate(fd_, map_struct->offset + map_struct->length) == -1))
        {
            file_size_ = required;
            // ...
        }
        const auto retr = ::mmap(
            nullptr,
            map_struct->length,
            PROT_READ | PROT_WRITE,
            MAP_SHARED,
            fd_,
            map_struct->offset);
        // ...
        return retr;
    }
    // ...
};
```

Process A

Process B

Race Condition

Two processes may attempt to unlink, both find nothing to unlink, and then both call open, thereafter they will have the same file open (and one will have truncated the other)

```
struct MemoryMappedFile {
    explicit MemoryMappedFile(const std::string& filename) : file_size_(0) {
        ::unlink(filename.c_str());
        fd_ = ::open(filename.c_str(), O_CREAT | O_TRUNC | O_RDWR, 0644)
        // ...
    }
    void* Map(const MapStruct& map_struct) {
        const auto required = map_struct->offset + map_struct->length;
        if (
            (required > file_size_) &&
            (::ftruncate(fd_, map_struct->offset + map_struct->length) == -1))
        {
            file_size_ = required;
            // ...
        }
        const auto retr = ::mmap(
            nullptr,
            map_struct->length,
            PROT_READ | PROT_WRITE,
            MAP_SHARED,
            fd_,
            map_struct->offset);
        // ...
        return retr;
    }
    // ...
};
```

Process B

Process A

Race Condition

Two processes may attempt to unlink, both find nothing to unlink, and then both call open, thereafter they will have the same file open (and one will have truncated the other)

```
struct MemoryMappedFile {
    explicit MemoryMappedFile(const std::string& filename) : file_size_(0) {
        ::unlink(filename.c_str());
        fd_ = ::open(filename.c_str(), O_CREAT | O_TRUNC | O_RDWR, 0644)
        // ...
    }
    void* Map(const MapStruct& map_struct) {
        const auto required = map_struct->offset + map_struct->length;
        if (
            (required > file_size_) &&
            (::ftruncate(fd_, map_struct->offset + map_struct->length) == -1))
        {
            file_size_ = required;
            // ...
        }
        const auto retr = ::mmap(
            nullptr,
            map_struct->length,
            PROT_READ | PROT_WRITE,
            MAP_SHARED,
            fd_,
            map_struct->offset);
        // ...
        return retr;
    }
    // ...
};
```

Process B

Process A

Race Condition

Two processes may attempt to unlink, both find nothing to unlink, and then both call open, thereafter they will have the same file open (and one will have truncated the other)

```
struct MemoryMappedFile {
    explicit MemoryMappedFile(const std::string& filename) : file_size_(0) {
        ::unlink(filename.c_str());
        fd_ = ::open(filename.c_str(), O_CREAT | O_TRUNC | O_RDWR, 0644);
        // ...
    }
    void* Map(const MapStruct& map_struct) {
        const auto required = map_struct->offset + map_struct->length;
        if (
            (required > file_size_) &&
            (::ftruncate(fd_, map_struct->offset + map_struct->length) == -1))
        {
            file_size_ = required;
            // ...
        }
        const auto retri = ::mmap(
            nullptr,
            map_struct->length,
            PROT_READ | PROT_WRITE,
            MAP_SHARED,
            fd_,
            map_struct->offset);
        // ...
        return retri;
    }
    // ...
};
```

Process B

Process A



TOCTOU

Time Of Check to Time Of Use

- In some sense the call to `unlink` “checks” that the file does not exist (by deleting it if it does exist or doing nothing if it doesn’t exist)
- Thereafter we “use” that information when we call `open` expecting it to create (rather than truncate) a file
- Between when we “check” and when we “use” the information we “checked” may change (i.e. the file may be created) invalidating our overall algorithm

commit 32c292e017ed3e424a2986d2c091a062e5db2e95

Author: Robert Leahy <robert@mayastreet.com>

Date: Thu Jun 6 20:23:12 2024 -0400

bp::sys::MemoryMappedFile: Race When Creating File

As originally formulated by e6bfa04aa701b5813d751fbd6b17fe507d198a06

bp::sys::MemoryMappedFile::Init, when called with bp::sys::

kMemoryMappedFileMode_CreateNew as its second parameter, invoked the open syscall with at least the flags:

- O_CREAT, and
- O_TRUNC

This resulted in:

- The target file being created if it did not already exist, otherwise
- The target file being opened and truncated to length zero

Note that in the second case no file is actually created. An existing file is opened and then truncated to zero length. This meant that if another process had that file memory mapped it would receive a SIGBUS when trying to access it as the mapped section of the file had become invalid (since the file's length was shortened).

7b881dc3e290f67cae54483820398412359f3872 tried to work around this by first unlinking the target file. This meant that if, at the time of the unlink syscall, another process had the file with the target name opened that file would be unlinked from the filesystem (thereby becoming an anonymous inode unless it had links from elsewhere in the filesystem) and thereafter the open syscall would create a new file and leave the file opened by the other process untruncated. Unfortunately this formulation contained a race condition:

1. Process A unlinks the target file, but there is nothing to unlink
2. Process B unlinks the target file, but there is nothing to unlink
3. Process B opens the target file, creating it
4. Process B maps memory from the target file, lengthening it
5. Process A opens the target file, truncating it
6. Process B attempts to access the memory mapped in 4 and receives SIGBUS

7b881dc3e290f67cae54483820398412359f3872 actually acknowledges this race condition in a comment but calls it "unavoidable," which is wrong.

Reproduced the above-described race condition in two different tests (one for bp::sys::MemoryMappedFile directly and another for bp::stat::MemoryMappedFileWriterStream since that is the context in which this bug was encountered) and fixed by using the following algorithm when bp::sys::kMemoryMappedFileMode_CreateNew is provided:

1. Attempt to open the target file with flags O_CREAT and O_EXCL, if that succeeds skip the remainder of these steps, otherwise
2. Create a file using mkstemp whose name is based on the target filename
3. Rename the above-created file to the target filename (thereby atomically unlinking it)

Note that the first step is technically unnecessary, it just simplifies the algorithm in the common case.

commit 32c292e017ed3e424a2986d2c091a062e5db2e95

Author: Robert Leahy <robert@mayastreet.com>

Date: Thu Jun 6 20:23:12 2024 -0400

bp::sys::MemoryMappedFile: Race When Creating File

As originally formulated by e6bfa04aa701b5813d751fbd6b17fe507d198a06

bp::sys::MemoryMappedFile::Init, when called with bp::sys::

kMemoryMappedFileMode_CreateNew as its second parameter, invoked the open syscall with at least the flags:

- O_CREAT, and
- O_TRUNC

This resulted in:

- The target file being created if it did not already exist, otherwise
- The target file being opened and truncated to length zero

Note that in the second case no file is actually created. An existing file is opened and then truncated to zero length. This meant that if another process had that file memory mapped it would receive a SIGBUS when trying to access it as the mapped section of the file had become invalid (since the file's length was shortened).

7b881dc3e290f67cae54483820398412359f3872 tried to work around this by first unlinking the target file. This meant that if, at the time of the unlink syscall, another process had the file with the target name opened that file would be unlinked from the filesystem (thereby becoming an anonymous inode unless it had links from elsewhere in the filesystem) and thereafter the open syscall would create a new file and leave the file opened by the other process untruncated. Unfortunately this formulation contained a race condition:

1. Process A unlinks the target file, but there is nothing to unlink
2. Process B unlinks the target file, but there is nothing to unlink
3. Process B opens the target file, creating it
4. Process B maps memory from the target file, lengthening it
5. Process A opens the target file, truncating it
6. Process B attempts to access the memory mapped in 4 and receives SIGBUS

7b881dc3e290f67cae54483820398412359f3872 actually acknowledges this race condition in a comment but calls it "unavoidable," which is wrong.

Reproduced the above-described race condition in two different tests (one for bp::sys::MemoryMappedFile directly and another for bp::stat::MemoryMappedFileWriterStream since that is the context in which this bug was encountered) and fixed by using the following algorithm when bp::

sys::MemoryMappedFileMode_CreateNew is provided.

1. Attempt to open the target file with flags O_CREAT and O_EXCL, if that succeeds skip the remainder of these steps, otherwise
2. Create a file using mkstemp whose name is based on the target filename
3. Rename the above-created file to the target filename (thereby atomically unlinking it)

Note that the first step is technically unnecessary, it just simplifies the algorithm in the common case.

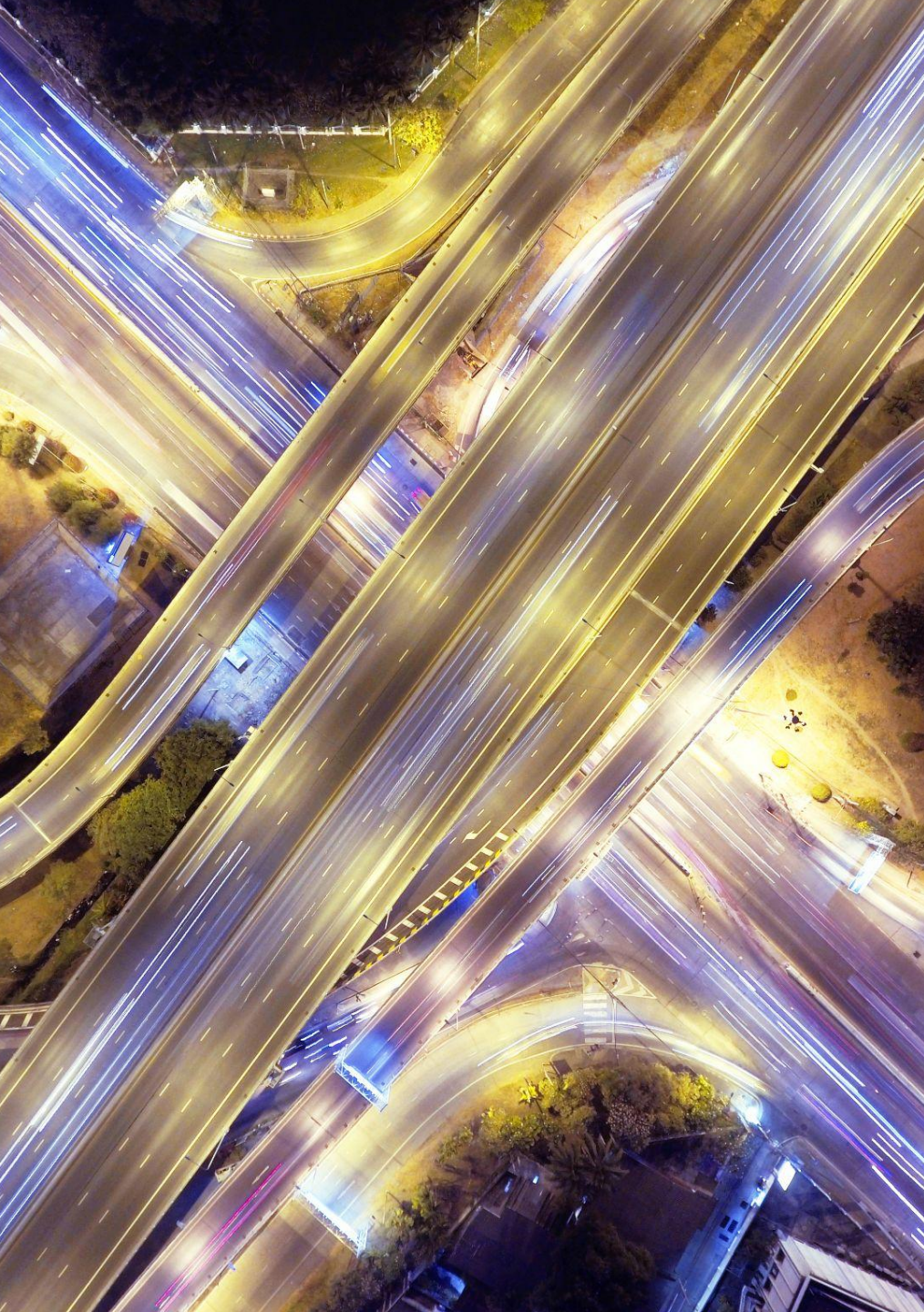
Test It

This approach seems hokey but it reliably reproduced the race

The test executable died reliably with SIGBUS before the fix

After the fix succeeds and this test case runs surprisingly quickly (less than a second)

```
TEST(MemoryMappedFile, CreateNewParallel) {
    const auto path = (tmpdir / "foo").string();
    std::vector<std::thread> threads;
    struct Guard {
        ~Guard() noexcept {
            while (!threads.empty()) {
                threads.back().join();
                threads.pop_back();
            }
        }
        std::vector<std::thread>& threads;
    };
    const Guard g{threads};
    for (unsigned u = 0; u < 100; ++u) {
        threads.emplace_back([&]() {
            for (unsigned u = 0; u < 100; ++u) {
                bp::sys::MemoryMappedFile file(path.c_str());
                const bp::sys::MapStruct map{0, 1024};
                const auto ptr = file.Map(map);
                std::memset(ptr, 1, 1024);
                // ...
            }
        });
    }
}
```



Double-Checked Locking

Lazy, thread-safe initialization

- Technique to lazily initialize an object to be shared by multiple threads
 - C++11 “magic statics” essentially added this to the core language
- Atomically check if object has been initialized, if it has been simply return it
 - This is the first check
- Otherwise acquire lock to perform initialization
 - Possible TOCTOU: What if someone else has performed the initialization since the check?
- Check again to see if object has been initialized, if it has simply return it
 - This is the second check
 - Resolves the above-described TOCTOU
- Perform initialization
- Return object

Metadata Retrieval

Function retrieves metadata for a data file

Metadata is retrieved lazily (first time it is requested) as an optimization

Instances of the class may be shared across threads

```
virtual const toml_table_ptr& metadata() const noexcept override {
    if (!_original_identifier.is_published()) {
        return _metadata;
    }
    {
        const _lock_guard g(_load_indices_lock);
        if (_metadata) {
            return _metadata;
        }
    }
    auto r = _parent->_get_metadata_for(_loaded_extents.fileh);
    if (!r) {
        return _metadata;
    }
    const _lock_guard g(_load_indices_lock);
    _metadata = std::move(r).value();
    return _metadata;
}
```

Race Condition

There are multiple data races in this implementation

Most significant is two threads loading the metadata and one clobbering the metadata loaded by the other

This can involve a bona fide data race (since one thread assigns to a value being referred to by another) but also a use-after-free since the last thread to assign frees memory being used by the first thread

```
virtual const toml_table_ptr& metadata() const noexcept override {
    if (!_original_identifer.is_published()) {
        return _metadata;
    }
    {
        const _lock_guard g(_load_indices_lock);
        if (_metadata) { ← Thread A
            return _metadata;
        }
    }
    auto r = _parent->_get_metadata_for(_loaded_extents.fileh);
    if (!r) {
        return _metadata;
    }
    const _lock_guard g(_load_indices_lock);
    _metadata = std::move(r).value();
    return _metadata;
}
```

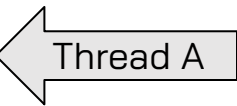
Race Condition

There are multiple data races in this implementation

Most significant is two threads loading the metadata and one clobbering the metadata loaded by the other

This can involve a bona fide data race (since one thread assigns to a value being referred to by another) but also a use-after-free since the last thread to assign frees memory being used by the first thread

```
virtual const toml_table_ptr& metadata() const noexcept override {
    if (!_original_identifier.is_published()) {
        return _metadata;
    }
    {
        const _lock_guard g(_load_indices_lock);
        if (_metadata) {
            return _metadata;
        }
        auto r = _parent->_get_metadata_for(_loaded_extents.fileh);
        if (!r) {
            return _metadata;
        }
        const _lock_guard g(_load_indices_lock);
        _metadata = std::move(r).value();
        return _metadata;
    }
}
```



Race Condition

There are multiple data races in this implementation

Most significant is two threads loading the metadata and one clobbering the metadata loaded by the other

This can involve a bona fide data race (since one thread assigns to a value being referred to by another) but also a use-after-free since the last thread to assign frees memory being used by the first thread

```
virtual const toml_table_ptr& metadata() const noexcept override {
    if (!_original_identfier.is_published()) {
        return _metadata;
    }
    {
        const _lock_guard g(_load_indices_lock);
        if (_metadata) { ← Thread B
            return _metadata;
        }
    }
    auto r = _parent->_get_metadata_for(_loaded_extents.fileh); ← Thread A
    if (!r) {
        return _metadata;
    }
    const _lock_guard g(_load_indices_lock);
    _metadata = std::move(r).value();
    return _metadata;
}
```

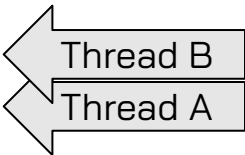
Race Condition

There are multiple data races in this implementation

Most significant is two threads loading the metadata and one clobbering the metadata loaded by the other

This can involve a bona fide data race (since one thread assigns to a value being referred to by another) but also a use-after-free since the last thread to assign frees memory being used by the first thread

```
virtual const toml_table_ptr& metadata() const noexcept override {
    if (!_original_identifier.is_published()) {
        return _metadata;
    }
    {
        const _lock_guard g(_load_indices_lock);
        if (_metadata) {
            return _metadata;
        }
        auto r = _parent->_get_metadata_for(_loaded_extents.fileh);
        if (!r) {
            return _metadata;
        }
        const _lock_guard g(_load_indices_lock);
        _metadata = std::move(r).value();
        return _metadata;
    }
}
```



Race Condition

There are multiple data races in this implementation

Most significant is two threads loading the metadata and one clobbering the metadata loaded by the other

This can involve a bona fide data race (since one thread assigns to a value being referred to by another) but also a use-after-free since the last thread to assign frees memory being used by the first thread

```
virtual const toml_table_ptr& metadata() const noexcept override {
    if (!_original_identifier.is_published()) {
        return _metadata;
    }
    {
        const _lock_guard g(_load_indices_lock);
        if (_metadata) {
            return _metadata;
        }
        auto r = _parent->_get_metadata_for(_loaded_extents.fileh);
        if (!r) {
            return _metadata;
        }
        const _lock_guard g(_load_indices_lock);
        _metadata = std::move(r).value();
        return _metadata;
    }
}
```

Thread B

Thread A

Race Condition

There are multiple data races in this implementation

Most significant is two threads loading the metadata and one clobbering the metadata loaded by the other

This can involve a bona fide data race (since one thread assigns to a value being referred to by another) but also a use-after-free since the last thread to assign frees memory being used by the first thread

```
virtual const toml_table_ptr& metadata() const noexcept override {  
    if (!_original_identfier.is_published()) {  
        return _metadata;  
    }  
    {  
        const _lock_guard g(_load_indices_lock);  
        if (_metadata) {  
            return _metadata;  
        }  
        auto r = _parent->_get_metadata_for(_loaded_extents.fileh);  
        if (!r) {  
            return _metadata;  
        }  
        const _lock_guard g(_load_indices_lock);  
        _metadata = std::move(r).value();  
        return _metadata;  
    }  
}
```

Thread B

Thread A

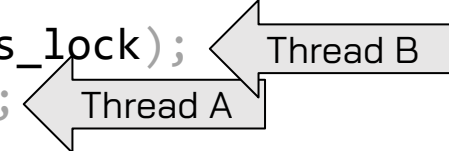
Race Condition

There are multiple data races in this implementation

Most significant is two threads loading the metadata and one clobbering the metadata loaded by the other

This can involve a bona fide data race (since one thread assigns to a value being referred to by another) but also a use-after-free since the last thread to assign frees memory being used by the first thread

```
virtual const toml_table_ptr& metadata() const noexcept override {
    if (!_original_identifier.is_published()) {
        return _metadata;
    }
    {
        const _lock_guard g(_load_indices_lock);
        if (_metadata) {
            return _metadata;
        }
        auto r = _parent->_get_metadata_for(_loaded_extents.fileh);
        if (!r) {
            return _metadata;
        }
        const _lock_guard g(_load_indices_lock);
        _metadata = std::move(r).value();
        return _metadata;
    }
}
```



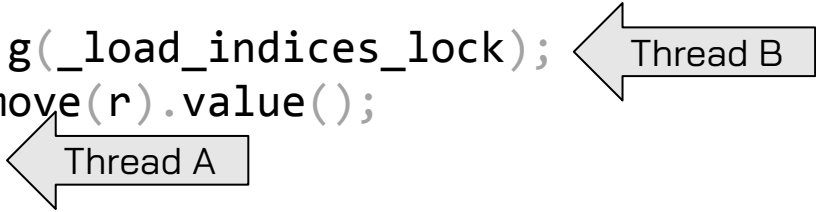
Race Condition

There are multiple data races in this implementation

Most significant is two threads loading the metadata and one clobbering the metadata loaded by the other

This can involve a bona fide data race (since one thread assigns to a value being referred to by another) but also a use-after-free since the last thread to assign frees memory being used by the first thread

```
virtual const toml_table_ptr& metadata() const noexcept override {
    if (!_original_identifer.is_published()) {
        return _metadata;
    }
    {
        const _lock_guard g(_load_indices_lock);
        if (_metadata) {
            return _metadata;
        }
        auto r = _parent->_get_metadata_for(_loaded_extents.fileh);
        if (!r) {
            return _metadata;
        }
        const _lock_guard g(_load_indices_lock);
        _metadata = std::move(r).value();
        return _metadata;
    }
}
```



Race Condition

There are multiple data races in this implementation

Most significant is two threads loading the metadata and one clobbering the metadata loaded by the other

This can involve a bona fide data race (since one thread assigns to a value being referred to by another) but also a use-after-free since the last thread to assign frees memory being used by the first thread

```
virtual const toml_table_ptr& metadata() const noexcept override {
    if (!_original_identifer.is_published()) {
        return _metadata;
    }
    {
        const _lock_guard g(_load_indices_lock);
        if (_metadata) {
            return _metadata;
        }
        auto r = _parent->_get_metadata_for(_loaded_extents.fileh);
        if (!r) {
            return _metadata;
        }
        const _lock_guard g(_load_indices_lock);
        _metadata = std::move(r).value();
        return _metadata;
    }
}
```

Thread B



Data Race

Undefined Behavior (UB)

- Since `metadata` returns `const tom1_table_ptr&` thread A continues reading therefrom outside the critical section
- Thread B enters the critical section and writes to the `tom1_table_ptr`
- Read and write without synchronization, therefore UB



Use-After-Free

More UB!

- Assigning to a `unique_ptr` deallocates the previously-managed object
- When thread B performs this allocation thread A may be examining the configuration tree
- Configuration tree is deallocated while thread A is using it, leading to a use-after-free

Core dump is not helpful

commit 41f6621dfe9c8cf401e264a40194c452d9c64cc5

Author: Robert Leahy <robert@maystreet.com>

Date: Fri Mar 25 18:23:13 2022 -0400

mdx::day_exchange_file_queryer::metadata: Races

The mdx::day_exchange_file_queryer::metadata function was previously implemented as follows:

1. Check if the backing DXF was originally unpublished, if so return empty metadata immediately
2. While holding a lock check to see if the metadata is already populated, if so return that metadata, otherwise release the lock
3. Attempt to read the metadata from the file system, if that fails return the (assumedly empty) metadata
4. While holding a lock set the metadata and return it

There were two races in this implementation. First:

1. Thread A performs the check in 2 and advances to 3
2. Thread B performs the check in 2 and advances to 3
3. Thread A advances to 4
4. Thread B advances to 4 thereby replacing and invalidating the metadata assigned by A (this is not only a data race since thread B is writing wherefrom thread A may be reading unsynchronized but can also result in a dangling pointer since thread B will free the TOML table on which thread A may be working)

Fixed by making the assignment conditional so that when thread B finishes loading the metadata rather than replacing the metadata left by thread A it adopts it and discards the metadata loaded by thread B.

Second:

1. Thread A performs 3, fails, and proceeds with empty metadata
2. Thread B performs 3, does not fail, advances to 4, and replaces the metadata which thread A has a reference to and from which it may be reading (i.e. a data race)

Fixed by using a Meyer's Singleton mdx::toml_table_ptr in 1 (arguably unnecessary) and when 3 fails.

This latter race was also present in mdx::day_exchange_file_queryer::day_metadata which has also been fixed.

Slight Variation on Previous Test

When testing the memory mapped file race we could simply create files continually in parallel with threads being created and starting immediately

This latest race condition only occurs on first access so we need to delay the start of the test until all the racing threads are ready (to increase odds of observing a problematic ordering)

More modern C++ would use `std::barrier` but this code was written under a pre-C++20 standard

```
BOOST_AUTO_TEST_CASE(validates / day_exchange_file_queryer / metadata / parallel,  
    "Queryer metadata can be retrieved from multiple threads "  
    "simultaneously without inducing a race condition") {  
    const auto mydb = test_database();  
    TRY(updater, mydb->update(new_test_identity()));  
    // ...  
    TRY(db, mdx::make_day_exchange_file_database(mydb->path_handle(), {}));  
    TRY(queryer, db->query(updater->identifier()));  
    std::vector<void*> ptrs(100);  
    std::condition_variable cv;  
    std::mutex l;  
    bool go = false;  
    const auto make_thread = [&](void*& ptr) {  
        return std::thread([&]() noexcept {  
            { std::unique_lock g(l); cv.wait(g, [&]() noexcept { return go; }); }  
            ptr = queryer->metadata().get();  
        });  
    };  
    {  
        std::vector<std::thread> ts;  
        const auto g = mdx::make_scope_exit([&]() noexcept {  
            { const std::lock_guard g(l); go = true; cv.notify_all(); }  
            for(auto&& t : ts) t.join();  
        });  
        for(auto&& ptr : ptrs) ts.push_back(make_thread(ptr));  
    }  
    BOOST_CHECK(ptrs.front() && std::all_of(  
        ptrs.begin() + 1,  
        ptrs.end(),  
        [&](const auto& ptr) noexcept { return ptr == ptrs.front(); }));  
}
```

Mailbox

Communicates between two threads using a front and back buffer

Producer writes into the front buffer, consumer reads from the back buffer

Producer “interlocks” when front buffer is full

Consumer “interlocks” when back buffer is empty

Once both threads “interlock” the buffers are swapped and they resume

```
template<typename Member>
class Mailbox {
private:
    mutable std::mutex access;
    mutable std::condition_variable ready;
    // ...
    bool outputExhausted() const { /* ... */ }
    void awaken(const std::unique_lock<std::mutex>&) { /* ... */ }
    void await(std::unique_lock<std::mutex>& lock) {
        suspended = true;
        ready.wait(lock);
    }
    void interlock(std::unique_lock<std::mutex>& lock) {
        if(suspended) awaken(lock);
        else await(lock);
    }
public:
    // ...
    Member pop() {
        if(outputExhausted()) {
            std::unique_lock lock(access);
            // ...
            interlock(lock);
            // ...
            assert(!output.empty());
        }
        return std::move(*next++);
    }
};
```

Mailbox

When pop is attempted and the front buffer is exhausted we interlock to synchronize and promote the back buffer

```
template<typename Member>
class Mailbox {
private:
    mutable std::mutex access;
    mutable std::condition_variable ready;
    // ...
    bool outputExhausted() const { /* ... */ }
    void awaken(const std::unique_lock<std::mutex>&) { /* ... */ }
    void await(std::unique_lock<std::mutex>& lock) {
        suspended = true;
        ready.wait(lock);
    }
    void interlock(std::unique_lock<std::mutex>& lock) {
        if(suspended) awaken(lock);
        else await(lock);
    }
public:
    // ...
    Member pop() {
        if(outputExhausted()) {
            std::unique_lock lock(access);
            // ...
            interlock(lock);
            // ...
            assert(!output.empty());
        }
        return std::move(*next++);
    }
};
```

Mailbox

If we find that the other thread isn't already waiting we need to wait for it

```
template<typename Member>
class Mailbox {
private:
    mutable std::mutex access;
    mutable std::condition_variable ready;
    // ...
    bool outputExhausted() const { /* ... */ }
    void awaken(const std::unique_lock<std::mutex>&) { /* ... */ }
    void await(std::unique_lock<std::mutex>& lock) {
        suspended = true;
        ready.wait(lock);
    }
    void interlock(std::unique_lock<std::mutex>& lock) {
        if(suspended) awaken(lock);
        else await(lock);
    }
public:
    // ...
    Member pop() {
        if(outputExhausted()) {
            std::unique_lock lock(access);
            // ...
            interlock(lock);
            // ...
            assert(!output.empty());
        }
        return std::move(*next++);
    }
};
```

Mailbox

We signal that we're waiting by setting `suspended` and then `wait` on the condition variable

Returning back up the call stack we `assert` that the output is not empty (i.e. the back buffer has been promoted)

This must surely be the case otherwise the producer would not have notified us, right?

```
template<typename Member>
class Mailbox {
private:
    mutable std::mutex access;
    mutable std::condition_variable ready;
    // ...
    bool outputExhausted() const { /* ... */ }
    void awaken(const std::unique_lock<std::mutex>&) { /* ... */ }
    void await(std::unique_lock<std::mutex>& lock) {
        suspended = true;
        ready.wait(lock);
    }
    void interlock(std::unique_lock<std::mutex>& lock) {
        if(suspended) awaken(lock);
        else await(lock);
    }
public:
    // ...
    Member pop() {
        if(outputExhausted()) {
            std::unique_lock lock(access);
            // ...
            interlock(lock);
            // ...
            assert(!output.empty());
        }
        return std::move(*next++);
    }
};
```

commit 975a599f8cba2d85601d83beb715fcff0b3187f4

Author: Robert Leahy <robert@maystreet.com>

Date: Fri Dec 29 13:50:51 2023 -0500

bp::tools::crow::Mailbox: Interlock Fix

The otherwise-independent producer and consumer of a bp::tools::crow::Mailbox synchronize during an interlock phase during which pending data is pushed from the producer to the consumer. This phase is not unilateral instead involving both threads arriving at the interlocking with one of them then performing an otherwise-unsafe operation.

Interlocking was previously implemented by the private member function bp::tools::crow::Mailbox::interlock which deferred to one of two additional private member functions depending on the current state:

- bp::tools::crow::Mailbox::awaken if the other side was currently suspended
- bp::tools::crow::Mailbox::await otherwise

The latter of these simply set the boolean flag indicating a thread was suspended and then waited on a condition variable.

When waiting on a condition variable spurious wakeups may occur (i.e. the wait operation may end despite there having been no corresponding notification). pthreads allows this:

"Spurious wakeups from the pthread_cond_wait() or pthread_cond_timedwait() functions may occur."

Source: [https://pubs.opengroup.org/onlinepubs/7908799/xsh/\[...\]](https://pubs.opengroup.org/onlinepubs/7908799/xsh/[...])

WinAPI allows this:

"Condition variables are subject to spurious wakeups (those not associated with an explicit wake) and stolen wakeups (another thread manages to run before the woken thread)."

Source: [https://learn.microsoft.com/en-us/windows/win32/sync/\[...\]](https://learn.microsoft.com/en-us/windows/win32/sync/[...])

std::condition_variable::wait and std::condition_variable_any::wait allow this:

"The function will unblock when signaled by a call to notify_one() or a

call to notify_all(), or spuriously."

Sources: [thread.condition.condvar] and [thread.condvarany.wait]

Et cetera. Adding a unit test (included in this commit) which induced spurious wakeups resulted in that test ending with an assertion of the form:

```
mailbox_test: /scratch/bellport/src/bp/tools/crow/test/./Mailbox.hpp:406:
...
Assertion `not this->output.empty()' failed.
```

Since bp::tools::crow::Mailbox::awaken cleared the suspend flag the seemingly-obvious solution was to simply predicate awakening on that flag having been cleared. This fixed the issue of spurious wakeups but introduced a race condition:

1. Consumer interlocks and suspends
2. Producer interlocks and awakens
3. Producer continues and attempts to interlock again by calling bp::tools::crow::Mailbox::interlock
4. Producer calls bp::tools::crow::Mailbox::await since the suspend flag has been cleared (by the producer in step 2)
5. Producer sets the suspend flag (since that is part of bp::tools::crow::Mailbox::await) and begins waiting for it to be cleared
6. Consumer finally awakens, checks the suspend flag, sees that it's still set, and continues waiting

The above was not only possible but was actually observed in testing. bp::tools::crow::Mailbox::interlock did not actually interlock. Entrance to the critical section was synchronized but that is insufficient to form an interlocking. Interlocking implies mutual dependence. Previously the first thread to arrive (which suspended itself) depended on the second thread to proceed, but the second thread did not depend on the first thread. The dependence was not mutual.

Reimplemented the interlocking mechanism so it's actually an interlocking consisting of three phases:

1. Enter: The first thread to arrive waits for the second to signal that it has arrived
2. Critical section: The producer pushes data to the consumer
3. Exit: Both threads wait to be signaled that the other has left the critical section before proceeding

commit 975a599f8cba2d85601d83beb715fcff0b3187f4

Author: Robert Leahy <robert@maystreet.com>

Date: Fri Dec 29 13:50:51 2023 -0500

bp::tools::crow::Mailbox: Interlock Fix

The otherwise-independent producer and consumer of a bp::tools::crow::Mailbox synchronize during an interlock phase during which pending data is pushed from the producer to the consumer. This phase is not unilateral instead involving both threads arriving at the interlocking with one of them then performing an otherwise-unsafe operation.

Interlocking was previously implemented by the private member function bp::tools::crow::Mailbox::interlock which deferred to one of two additional private member functions depending on the current state:

- bp::tools::crow::Mailbox::awaken if the other side was currently suspended
- bp::tools::crow::Mailbox::await otherwise

The latter of these simply set the boolean flag indicating a thread was suspended and then waited on a condition variable.

When waiting on a condition variable spurious wakeups may occur (i.e. the wait operation may end despite there having been no corresponding notification). pthreads allows this:

"Spurious wakeups from the pthread_cond_wait() or pthread_cond_timedwait() functions may occur."

Source: [https://pubs.opengroup.org/onlinepubs/7908799/xsh/\[...\]](https://pubs.opengroup.org/onlinepubs/7908799/xsh/[...])

WinAPI allows this:

"Condition variables are subject to spurious wakeups (those not associated with an explicit wake) and stolen wakeups (another thread manages to run before the woken thread)."

Source: [https://learn.microsoft.com/en-us/windows/win32/sync/\[...\]](https://learn.microsoft.com/en-us/windows/win32/sync/[...])

std::condition_variable::wait and std::condition_variable_any::wait allow this:

"The function will unblock when signaled by a call to notify_one() or a

call to notify_all(), or spuriously."

Sources: [thread.condition.condvar] and [thread.condvarany.wait]

Et cetera. Adding a unit test (included in this commit) which induced spurious wakeups resulted in that test ending with an assertion of the form:

```
mailbox_test: /scratch/bellport/src/bp/tools/crow/test/./Mailbox.hpp:406:
...
Assertion `not this->output.empty()' failed.
```

Since bp::tools::crow::Mailbox::awaken cleared the suspend flag the seemingly-obvious solution was to simply predicate awakening on that flag having been cleared. This fixed the issue of spurious wakeups but introduced a race condition:

1. Consumer interlocks and suspends
2. Producer interlocks and awakens
3. Producer continues and attempts to interlock again by calling bp::tools::crow::Mailbox::interlock
4. Producer calls bp::tools::crow::Mailbox::await since the suspend flag has been cleared (by the producer in step 2)
5. Producer sets the suspend flag (since that is part of bp::tools::crow::Mailbox::await) and begins waiting for it to be cleared
6. Consumer finally awakens, checks the suspend flag, sees that it's still set, and continues waiting

The above was not only possible but was actually observed in testing. bp::tools::crow::Mailbox::interlock did not actually interlock. Entrance to the critical section was synchronized but that is insufficient to form an interlocking. Interlocking implies mutual dependence. Previously the first thread to arrive (which suspended itself) depended on the second thread to proceed, but the second thread did not depend on the first thread. The dependence was not mutual.

Reimplemented the interlocking mechanism so it's actually an interlocking consisting of three phases:

1. Enter: The first thread to arrive waits for the second to signal that it has arrived
2. Critical section: The producer pushes data to the consumer
3. Exit: Both threads wait to be signaled that the other has left the critical section before proceeding

Modified Mailbox

Second template parameter allows the types of internal synchronization primitives to be customized

```
template<typename Member, typename ThreadingPackage>
class Mailbox {
private:
    using mutex_type = typename ThreadingPackage::mutex_type;
    using condition_variable_type =
        typename ThreadingPackage::condition_variable_type;
    mutable mutex_type access;
    mutable condition_variable_type ready;
    // ...
    bool outputExhausted() const { /* ... */ }
    void awaken(const std::unique_lock<mutex_type>&) { /* ... */ }
    void await(std::unique_lock<mutex_type>& lock) { /* ... */ }
    void interlock(std::unique_lock<mutex_type>& lock) { /* ... */ }
public:
    // ...
    Member pop() { /* ... */ }
};
```

Null Condition Variable

Since condition variables are permitted to wake up spuriously an implementation which does nothing is permissible

```
struct NullThreadingPackage {  
    using mutex_type = std::mutex;  
    struct condition_variable_type {  
        void notify_all() noexcept {}  
        template<typename Lock>  
        void wait(std::unique_lock<Lock>&) noexcept {}  
    };  
};
```

Null Condition Variable

The old code doesn't use the binary overload of wait but since our new code aims to be correct and modern it will require that overload be provided

```
struct NullThreadingPackage {  
    using mutex_type = std::mutex;  
    struct condition_variable_type {  
        void notify_all() noexcept {}  
        template<typename Lock>  
        void wait(std::unique_lock<Lock>&) noexcept {}  
        template<typename Lock, typename Predicate>  
        void wait(std::unique_lock<Lock>& l, Predicate should_stop) {  
            while (!should_stop()) {  
                l.unlock();  
                std::this_thread::yield();  
                l.lock();  
            }  
        }  
    };  
};
```

Null Condition Variable

In fact since we're aiming to be correct and modern we can ensure that our new implementation doesn't use the unary, bug-prone overload of wait by simply removing it

```
struct NullThreadingPackage {  
    using mutex_type = std::mutex;  
    struct condition_variable_type {  
        void notify_all() noexcept {}  
    };  
  
    template<typename Lock, typename Predicate>  
    void wait(std::unique_lock<Lock>& l, Predicate should_stop) {  
        while (!should_stop()) {  
            l.unlock();  
            std::this_thread::yield();  
            l.lock();  
        }  
    }  
};
```

But wait, there's more!

commit 85cbb20a31d76d301c48ba4bd493097f2993ac5e

Author: Robert Leahy <robert@maystreet.com>

Date: Mon Jan 22 10:43:46 2024 -0500

bp::tools::crow::Mailbox: Interlock Exception Safety

std::condition_variable::notify_one is noexcept. std::
condition_variable::wait:

- Does not throw in the case of the unary overload
- Throws only if the predicate throws in the case of the binary overload

Due to the presence of "interruption points" (which may throw an exception to indicate interruption) bp::tools::crow::threading_package::condition_variable does not follow the standard library's example in either of the above cases. The possibility the aforementioned functions throwing means that the interlock phase of bp::tools::crow::Mailbox may exit with an exception. This possibility was not considered when that phase was fixed/reformulated in 975a599f8cba2d85601d83beb715fcff0b3187f4.

Added a unit test which causes the condition variable used by an instance of bp::tools::crow::Mailbox to throw in each possible location and then updated the implementation of bp::tools::crow::Mailbox's interlock phase so that it tolerates such exceptions.

Change-Id: I9fba55d0c7b126c3516a9bffe30e86104762dd0d

commit 85cbb20a31d76d301c48ba4bd493097f2993ac5e

Author: Robert Leahy <robert@maystreet.com>

Date: Mon Jan 22 10:43:46 2024 -0500

bp::tools::crow::Mailbox: Interlock Exception Safety

std::condition_variable::notify_one is noexcept. std::
condition_variable::wait:

- Does not throw in the case of the unary overload
- Throws only if the predicate throws in the case of the binary overload

Due to the presence of "interruption points" (which may throw an exception to indicate interruption) bp::tools::crow::threading_package::condition_variable does not follow the standard library's example in either of the above cases. The possibility the aforementioned functions throwing means that the interlock phase of bp::tools::crow::Mailbox may exit with an exception. This possibility was not considered when that phase was fixed/reformulated in
975a599f8cba2d85601d83beb715fcff0b3187f4.

Added a unit test which causes the condition variable used by an instance of bp::tools::crow::Mailbox to throw in each possible location and then updated the implementation of bp::tools::crow::Mailbox's interlock phase so that it tolerates such exceptions.

Change-Id: I9fba55d0c7b126c3516a9bffe30e86104762dd0d

Throwing Condition Variable

Allows the throwing behavior of a mock condition variable to be customized

Each time it's interacted with it decrements a counter, once the counter reaches zero it throws and then never throws again

```
struct ThrowingThreadingPackage {  
    using mutex_type = std::mutex;  
    struct Exception : std::runtime_error {  
        using std::runtime_error::runtime_error;  
    };  
    template<typename Function>  
    static decltype(auto) AccessThrowsAfter_(Function f) {  
        static std::size_t throws_after{0};  
        static std::mutex access;  
        const std::lock_guard g(access);  
        return std::invoke(std::move(f), throws_after);  
    }  
    static void MaybeThrow_() {  
        AccessThrowsAfter_([](std::size_t& throws_after) {  
            if (throws_after) {  
                --throws_after;  
                return;  
            }  
            throws_after = std::numeric_limits<std::size_t>::max();  
            throw Exception("Throwing as requested");  
        }));  
    }  
    // ...  
};
```

Throwing Condition Variable

Allows the throwing behavior of a mock condition variable to be customized

Each time it's interacted with it decrements a counter, once the counter reaches zero it throws and then never throws again

```
struct ThrowingThreadingPackage {  
    // ...  
public:  
    template<typename Function>  
    static decltype(auto) Loop(bool& threw, Function f) {  
        for (;;) {  
            try {  
                return std::invoke(f);  
            } catch (const Exception&) {  
                threw = true;  
            }  
        }  
    }  
    static void ThrowsAfter(const std::size_t after) {  
        AccessThrowsAfter_([&](std::size_t& out) noexcept {  
            out = after;  
        });  
    }  
    struct condition_variable_type : std::condition_variable {  
        void notify_all() noexcept {  
            MaybeThrow_();  
            std::condition_variable::notify_all();  
        }  
        template<typename Lock, typename Predicate>  
        void wait(std::unique_lock<Lock>& l, Predicate should_stop) {  
            MaybeThrow_();  
            std::condition_variable::wait(l, std::move(should_stop));  
        }  
    };  
};
```

Test

Each loop iteration we throw one step later

Consumer thread is spawned and simply pops elements and pushes them in a `std::vector`

`FinishedException` is thrown when the producer finalizes the queue and causes the consumer thread to end

```
using MailboxType = Mailbox<int, ThrowingThreadingPackage>;
for (std::size_t throw_after = 0; ++throw_after) {
    ThrowingThreadingPackage::ThrowsAfter(throw_after);
    bool threw = false;
    std::exception_ptr ex;
    std::vector<int> actual;
    {
        MailboxType mailbox;
        std::thread t([&]() noexcept {
            try {
                for (;;) {
                    actual.push_back(
                        ThrowingThreadingPackage::Loop(
                            threw,
                            [&]() { return mailbox.pop(); }));
                }
            } catch (const MailboxType::FinishedException&) {
                // Pass
            } catch (...) {
                ex = std::current_exception();
            }
        });
        // ...
    }
}
```

Test

We ensure the queue is finalized and the thread joined when we exit the scope

Then we produce 100 sequential integers

At scope end everything ends and then we ensure nothing went wrong and that the expected 100 integers were produced

Once we go through a run where no exception is thrown, the test ends (we've ensured exceptions are handled correctly at every step)

```
// ...
const scope_exit g([&]()) {
    ThrowingConditionPrologue::Loop(
        threw,
        [&]() { mailbox.finish(); });
    t.join();
});
for (int i = 0; i < 100; ++i) {
    ThrowingThreadingPackage::Loop(
        threw,
        [&]() { mailbox.push(i); });
}
test.expect(!ex);
for (int i = 0; i < 100; ++i) {
    test.demand_ne(0, actual.size());
    test.expect_eq(i, actual.front());
    actual.erase(actual.begin());
}
test.expect(actual.empty());
if (!threw) {
    break;
}
}
```



All of this is local to the machine

What about the network?



Composed Async Operations

10

When asynchronous algorithms are constructed from calls to other initiating functions, the result is called a composed operation. [...] Layers of composition may be added to arbitrary degree, permitting algorithms of significant complexity to be developed.

—Vinnie Falco (P1100R0)

- Patterns from Networking TS enable composition
- Consumption of such operations simplifies development



ROBERT LEAHY

The Networking TS
in Practice: Testable,
Composable Asynchronous
I/O in C++

What about the cloud?



Accessing S3

...and testing it!

libs3

<https://github.com/bji/libs3>

<https://github.com/MauStreet/libs3> (our fork)

libcurl

<https://curl.se/libcurl/>



LocalStack

Mocking S3

<https://www.localstack.cloud/>

Local Development

Developers can easily start a LocalStack Docker container and obtain their own private S3

Continuous Integration

Each CI job receives its own private LocalStack container which exists for the lifetime of the job

IP address of the container is injected via environment variable which unit tests consult

LocalStack S3 is automatically populated with test buckets and keys based on submodule of the repository under test

Since the instance is private if tests need to mutate the contents thereof they can create, delete, et cetera buckets at will



Nothing is perfect

commit ffa2794147f3cdbebe07444cacea2b2c447ac361

Author: Robert Leahy <robert@maystreet.com>

Date: Wed Aug 30 20:52:14 2023 -0400

MultipartUploadWithFallback: HTTP/2 & LocalStack

Versions of libcurl newer than 7.62.0 will attempt to use HTTP/2 for TLS connections (see: https://curl.se/libcurl/c/CURLOPT_HTTP_VERSION.html). Unfortunately it seems that on tests with multiple simultaneous uploads LocalStack causes spurious failures when HTTP/2 is used.

To workaround the above added a new setting for all operations defined within bp::s3::S3: disable_http_2. When true (default is false) libcurl will be configured to only use HTTP/1.1. Exposed this new setting as a per key setting for bp::s3::MultipartUploadWithFallback and set it to true for all bp::s3::MultipartUploadWithFallback unit tests.

Change-Id: Ieae005e97c9ef025b1dc94e1654187dde2d49372

CURLOPT_HTTP_VERSION explained

Name

CURLOPT_HTTP_VERSION - HTTP protocol version to use

Synopsis

```
#include <curl/curl.h>
```

```
CURLcode curl\_easy\_setopt(CURL *handle, CURLOPT_HTTP_VERSION, long version);
```

Description

Pass *version* a long, set to one of the values described below. They ask libcurl to use the specific HTTP versions.

Note that the HTTP version is just a request. libcurl still prioritizes to reuse existing connections so it might then reuse a connection using an HTTP version you have not asked for.

Related:

[easy options](#)

[getinfo options](#)

[multi options](#)

[File a bug about this page](#)

[View manpage source](#)

commit 8ec175d1798b93e96fedc1f753017dce1e4d8851

Author: Robert Leahy <robert@maystreet.com>

Date: Sat Feb 25 12:13:01 2023 -0500

S3_create_request_context_ex2

S3_create_request_context_ex allows an S3SetupCurlCallback to be associated with a newly-created S3RequestContext. Immediately after any request running against that S3RequestContext initializes a libcurl easy handle that function will be invoked and therefore be given an opportunity to perform some action (usually customizing the options associated with the easy handle). S3SetupCurlCallback is defined as:

```
typedef S3Status (*S3SetupCurlCallback)(void *curlMulti, void *curlEasy,  
                                         void *setupData);
```

Where setupData is the setupCurlCallbackData supplied as the final argument to S3_create_request_context_ex.

While the ability to customize the options of any given easy handle is convenient and enables certain workflows an S3RequestContext is a multiplexer: It potentially handles many S3 requests and initializes easy handles for all of them. Because the only opaque pointer supplied to any S3SetupCurlCallback is the one supplied to S3_create_request_context_ex this means that from within the S3SetupCurlCallback one can't tell which request against the S3RequestContext is initializing a certain easy handle in pursuit of and therefore it's difficult, impossible, or unwieldy to configure the easy handles differentially (for example one might want certain connections to have CURLOPT_VERIFYPEER and CURLOPT_VERIFYHOST disabled but not others).

To enable the above workflow added a new function:

S3_create_request_context_ex2 which accepts an S3SetupCurlCallbackEx which is defined as:

```
typedef S3Status (*S3SetupCurlCallbackEx)(void *curlMulti, void *curlEasy,  
                                           void *setupData, void *requestData);
```

Where requestData is the opaque pointer supplied when initiating the corresponding operation.

What about those tests?

commit 4659f980e6d0095e54c281b963fda74638abe1f6

Author: Robert Leahy <robert@maystreet.com>

Date: Sun Feb 26 21:39:08 2023 -0500

bp::s3::S3: CURLOPT_VERBOSE

Leveraging functionality added by
ee1d59d920ffd79e014572c02c2de22175b46ca2 all operations provided as
subclasses of bp::s3::S3 now provide SettingsType::verbose which
defaults to false and which, when true, causes CURLOPT_VERBOSE to be
enabled for all libcurl easy handles created in pursuit of that
operation.

Change-Id: Iff52197b2e97d0989e44416f55ea517420bfe480

Test

Lists the contents of a test bucket (in LocalStack, configuration therefor is abstracted away by the Settings function)

```
TEST(S3, ListBucketVerboseAndDisableHttp2) {
    bp::s3::S3 s3;
    const auto ctx = BucketContext();
    auto settings = Settings();
    settings.verbose = true;
    settings.disable_http_2 = true;
    MockStdError err;
    bp::s3::S3::ListBucket list(&ctx, "", "", 1024, 5000, s3,
        std::move(settings));
    std::vector<std::string> keys;
    for (auto item = list.Next(); item; item = list.Next()) {
        keys.push_back(std::move(item->key));
    }
    const auto& str = err.Get();
    EXPECT_NE("", str);
    EXPECT_EXCLUDES(str, "HTTP/2");
    EXPECT_LE(2, keys.size());
    EXPECT_EQ(1, std::count(keys.begin(), keys.end(),
        "sfn1p3/cts_pillar/2022/05/"
        "05/sfn1p3_cts_pillar_20220505-08-Incr-A-0078.pcap.lz4"));
    EXPECT_EQ(1, std::count(keys.begin(), keys.end(),
        "sfn1p4/cts_pillar/2022/05/"
        "05/sfn1p4_cts_pillar_20220505-20-Incr-A-0078.pcap.lz4"));
}
```

Mock Standard Error?

The most interesting part of this test is the fact that we mock/intercept the standard error stream and then use that to assert that it:

- Isn't empty, and
- Doesn't indicate that the request was made with HTTP/2

```
TEST(S3, ListBucketVerboseAndDisableHttp2) {
    bp::s3::S3 s3;
    const auto ctx = BucketContext();
    auto settings = Settings();
    settings.verbose = true;
    settings.disable_http_2 = true;
    MockStdError err;
    bp::s3::S3::ListBucket list(&ctx, "", "", 1024, 5000, s3,
        std::move(settings));
    std::vector<std::string> keys;
    for (auto item = list.Next(); item; item = list.Next()) {
        keys.push_back(std::move(item->key));
    }
    const auto& str = err.Get();
    EXPECT_NE("", str);
    EXPECT_EXCLUDES(str, "HTTP/2");
    EXPECT_EQ(2, keys.size());
    EXPECT_EQ(1, std::count(keys.begin(), keys.end(),
        "sfn1p3/cts_pillar/2022/05/"
        "05/sfn1p3_cts_pillar_20220505-08-Incr-A-0078.pcap.lz4"));
    EXPECT_EQ(1, std::count(keys.begin(), keys.end(),
        "sfn1p4/cts_pillar/2022/05/"
        "05/sfn1p4_cts_pillar_20220505-20-Incr-A-0078.pcap.lz4"));
}
```

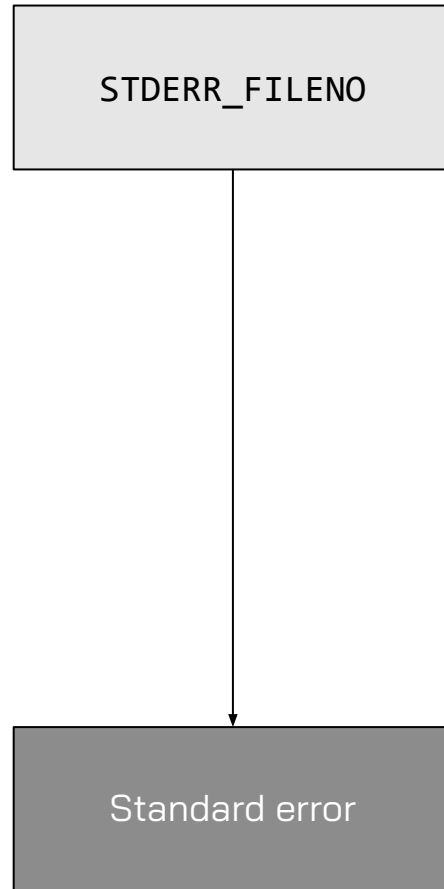
Mock Standard Error

From the completion of the constructor until the first call to `Cleanup_` standard error is redirected to the write end of a pipe

```
class MockStdError {
    void Cleanup_() noexcept {
        if (read_ == -1) return;
        ::dup2(fd_, STDERR_FILENO);
        ::close(read_);
        read_ = -1;
        ::close(fd_);
    }
    int read_, fd_;
public:
    MockStdError() : fd_(::dup(STDERR_FILENO)) {
        if (fd_ < 0) throw std::runtime_error("dup failed");
        bp::sys::CloseGuard g(fd_);
        int pipes[2];
        if (::pipe(pipes) < 0) throw std::runtime_error("pipe failed");
        bp::sys::CloseGuard g1(pipes[0]);
        const bp::sys::CloseGuard g2(pipes[1]);
        read_ = pipes[0];
        int flags = ::fcntl(read_, F_GETFL);
        if (flags < 0) throw std::runtime_error("fcntl(..., F_GETFL) failed");
        flags |= O_NONBLOCK;
        if (::fcntl(read_, F_SETFL, flags) < 0) throw std::runtime_error("...");
        if (::dup2(pipes[1], STDERR_FILENO) < 0) throw std::runtime_error("...");
        g1.Release();
        g.Release();
    }
    ~MockStdError() noexcept { Cleanup_(); }
    // ...
};
```

Mock Standard Error

Before we enter the constructor of MockStdError

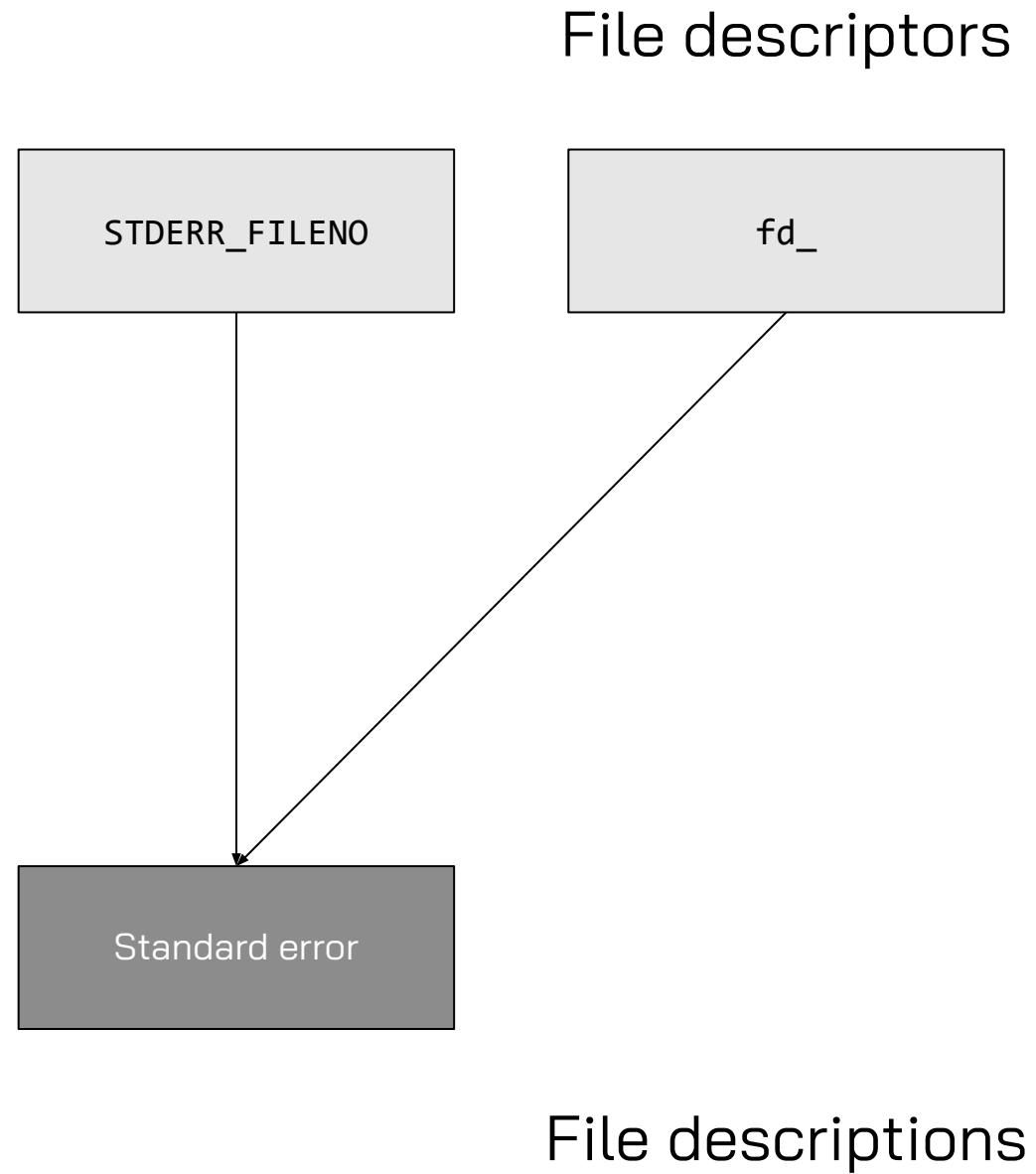


File descriptors

File descriptions

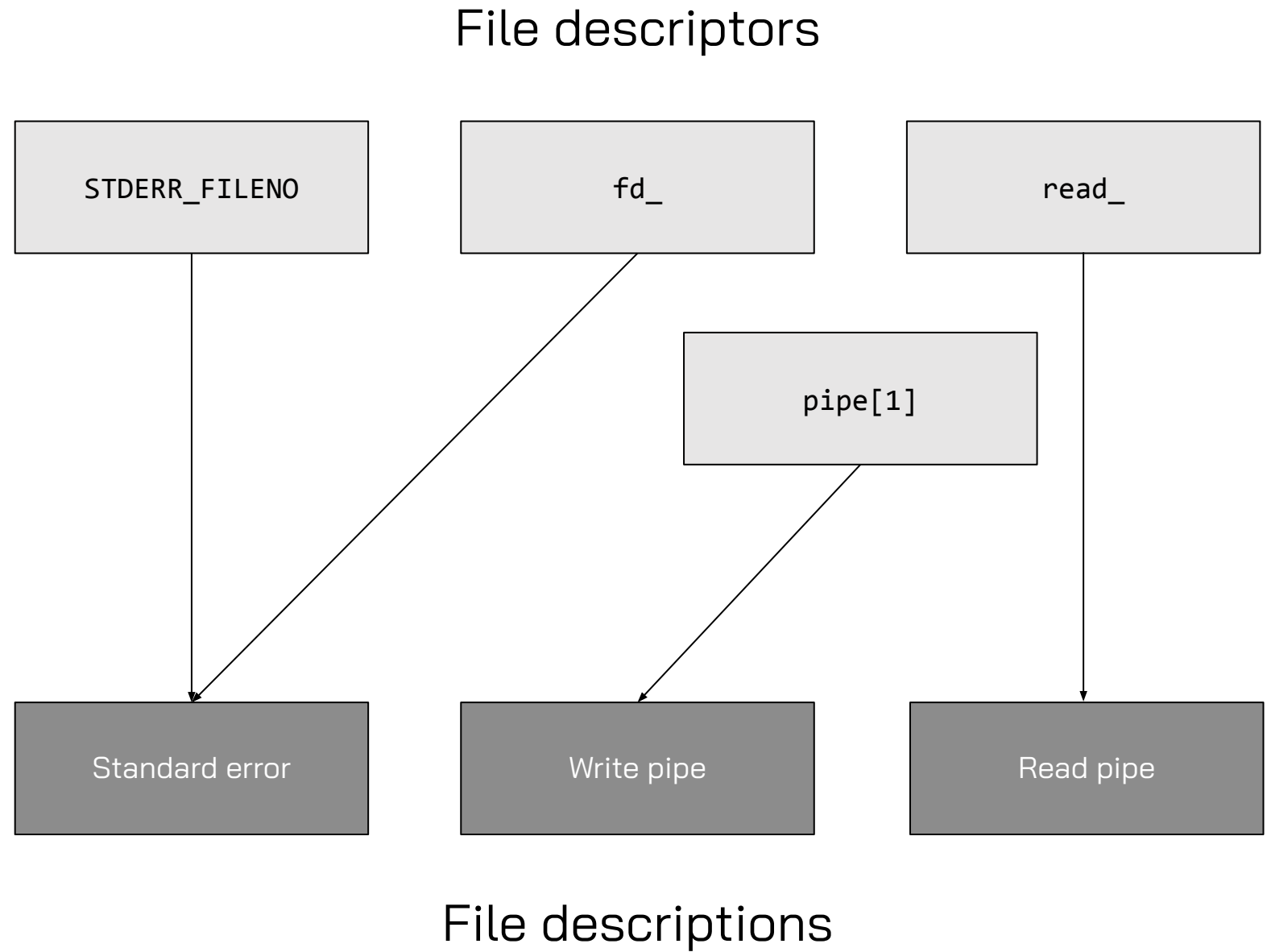
Mock Standard Error

After the call to dup in the constructor's initializer list



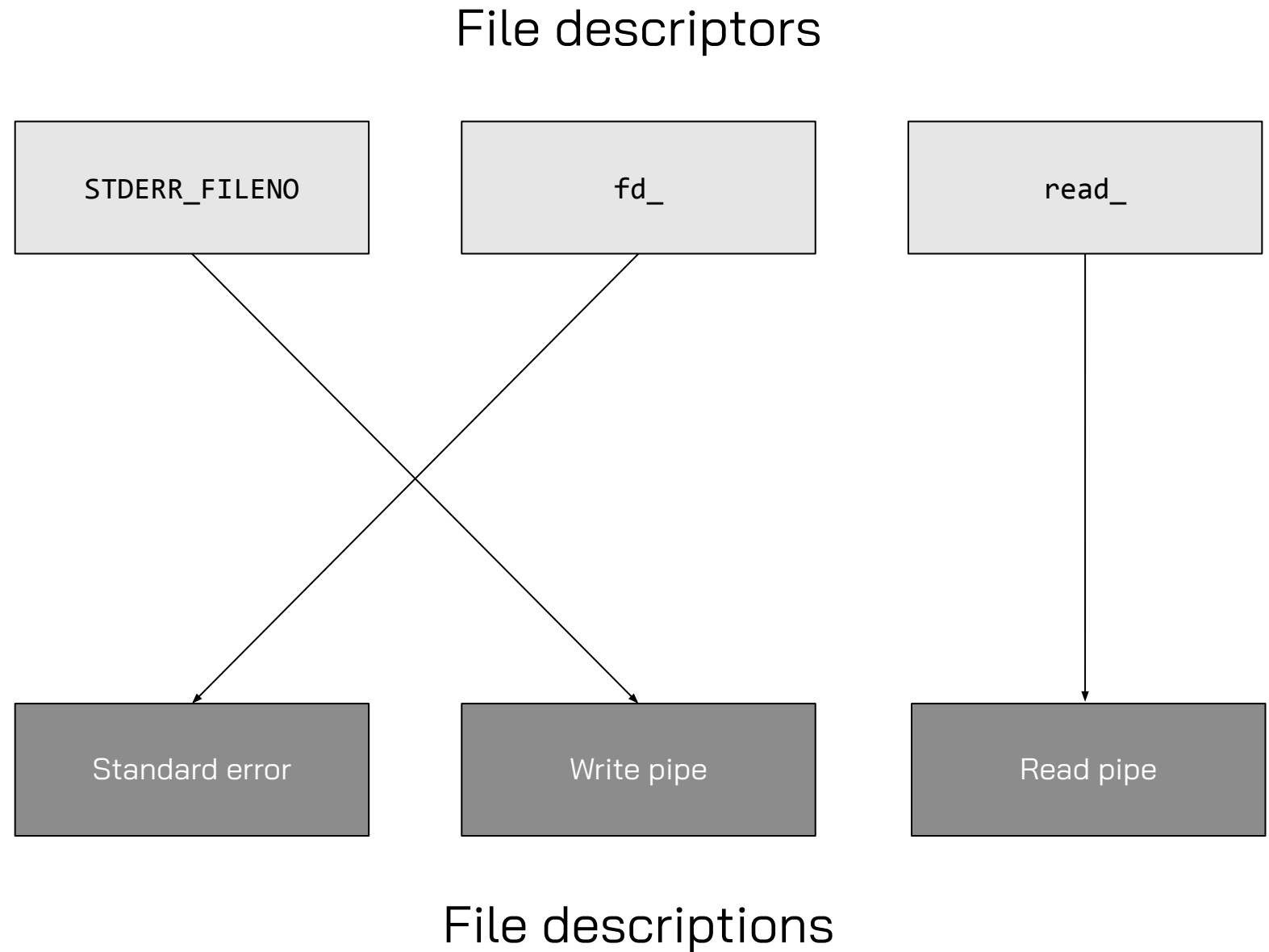
Mock Standard Error

After calling pipe



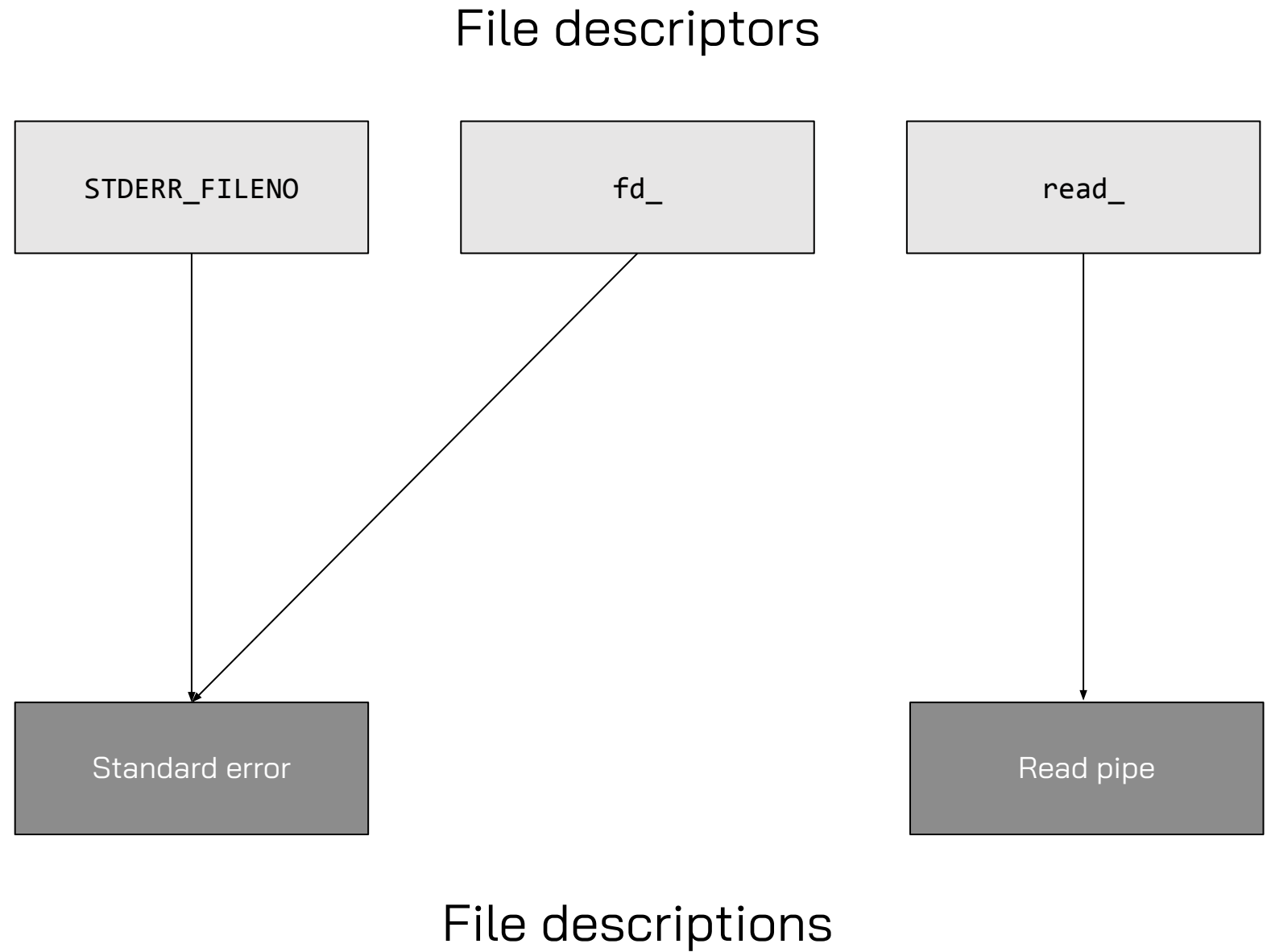
Mock Standard Error

After calling dup2 and close on pipe[1]



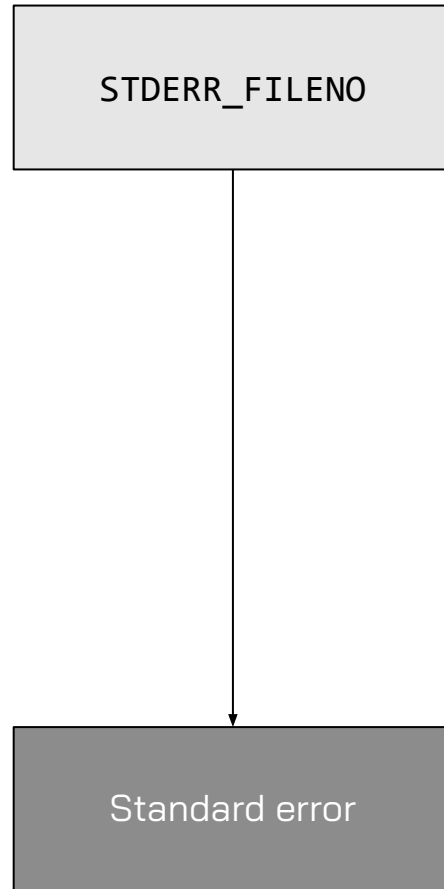
Mock Standard Error

Moving on to Cleanup_
After calling dup2



Mock Standard Error

After two calls to close



File descriptors

File descriptions

Mock Standard Error

This function extracts everything written to the pipe and passes it to a provided function for inspection

Thereafter everything is cleaned up so any subsequent writes to standard error will “leak” and be observed in logs et cetera

```
class MockStdError {
// ...
std::string Get() {
    std::string str;
    std::string::size_type size(0);
    for (;;) {
        str.resize(size + 1024);
        const auto read = ::read(read_, str.data() + size, str.size() - size);
        if (read < 0) {
            if ((errno == EWOULDBLOCK) || (errno == EAGAIN)) {
                str.resize(size);
                break;
            }
            throw std::runtime_error("read failed");
        }
        if (!read) {
            break;
        }
        size += read;
        str.resize(size);
    }
    Cleanup_();
    return str;
}
};
```

Some Code is Too Trivial, Right?

Simple helper function which accepts a C array and returns the string contents thereof

Use case is reading arrays from the wire which are padded with null bytes

```
template<std::size_t N>  
std::string_view ToStringView(const char (&arr)[N]) noexcept;
```

Oops!

I actually wrote the code on the right

```
template<std::size_t N>
std::string_view ToStringView(const char (&arr)[N]) noexcept {
    std::string_view retr(arr, N);
    while (!retr.empty() && !retr.back()) {
        retr.remove_suffix(1);
    }
    // ???
}
```

Fortunately Obvious

The bug seems obvious by inspection, but given that it was actually written the bug is obviously plausible

```
template<std::size_t N>
std::string_view ToStringView(const char (&arr)[N]) noexcept {
    std::string_view retr(arr, N);
    while (!retr.empty() && !retr.back()) {
        retr.remove_suffix(1);
    }
    return retr;
}
```

Tests

Nothing is too simple to bring under test

```
TEST(ToStringView, Full) {  
    const char arr[] = {'f', 'o', 'o'};  
    EXPECT_EQ("foo", ToStringView(arr));  
}
```

```
TEST(ToStringView, NullTerminated) {  
    const char arr[] = "foo";  
    static_assert(sizeof(arr) == 4U);  
    EXPECT_EQ("foo", ToStringView(arr));  
}
```

```
TEST(ToStringView, Empty) {  
    const char arr[4] = {};  
    EXPECT_EQ("", ToStringView(arr));  
}
```



Summary

You should write unit tests!

- Don't write unit tests because of delivered wisdom
- Write unit tests because it
 - Levers up your output
 - Improves your design
 - Improves your code
- Where there's a will there's a way
 - Once you decide you can't not test you'll find a way
- No component or bug is too small, too simple, or too trivial to be tested

Thank you

Robert Leahy
Lead Software Engineer
rleahy@rleahy.ca



LSEG

Questions?



LSEG